[4] W. Y. Wang, M. L. Chan, T. T. Lee, and C. H. Liu, "Adaptive fuzzy control for strict-feedback canonical nonlinear systems with H_∞ tracking performance," *IEEE Trans. Syst. Man Cybern. B, Cybern.*, vol. 30, no. 6, pp. 878–885, Dec. 2000.
[5] S. S. Ge, C. C. Hang, T. H. Lee, and T. Zhang, *Stable Adaptive Neural Network Control*. Boston, MA: Kluwer, 2001.
[6] C. F. Hsu, C. M. Lin, and T. T. Lee, "Wavelet adaptive backstepping control for a class of nonlinear systems," *IEEE Trans. Neural Netw.*, vol. 17, no. 5, pp. 1175–1183, Sep. 2006.
[7] D. Wang and J. Huang, "Neural network-based adaptive dynamic surface control for a class of uncertain nonlinear systems in strict-feedback form," *IEEE Trans. Neural Netw.*, vol. 16, no. 1, pp. 195–202, Jan. 2005.
[8] H. Du, H. Shao, and P. Yao, "Adaptive neural network control for a class of low-triangular-structured nonlinear systems," *IEEE Trans. Neural Netw.*, vol. 17, no. 2, pp. 509–514, Mar. 2006.
[9] S. S. Ge, F. Hong, and T. H. Lee, "Adaptive neural control of nonlinear time-delay systems with unknown virtual control coefficients," *IEEE Trans. Syst. Man Cybern. B, Cybern.*, vol. 34, no. 1, pp. 499–516, Feb. 2004.
[10] W. Chen, "Adaptive NN control for discrete-time pure-feedback systems with unknown control direction under amplitude and rate actuator constrains," *ISA Trans.*, vol. 48, no. 3, pp. 304–311, Jul. 2009.
[11] W. Chen and J. Li, "Decentralized output-feedback neural control for systems with unknown interconnections," *IEEE Trans. Syst. Man Cybern. B, Cybern.*, vol. 38, no. 1, pp. 258–266, Feb. 2008.
[12] S. Hara, Y. Yamamoto, T. Omata, and M. Nakano, "Repetitive control system: A new type servo system for peiodic exogenous signals," *IEEE Trans. Autom. Control*, vol. AC-33, no. 7, pp. 258–266, Jul. 1988.
[13] Y. P. Tian and X. Yu, "Robust learning control for a class of nonlinear systems with periodic and aperiodic uncertainties," *Automatica*, vol. 39, no. 11, pp. 1957–1966, Nov. 2003.
[14] J. X. Xu, "A new periodic adaptive control approach for time-varying parameters with known periodicity," *IEEE Trans. Autom. Control*, vol. 49, no. 4, pp. 579–583, Apr. 2004.
[15] Z. Ding, "Adaptive estimation and rejection of unknown sinusoidal disturbances in a class of non-minimum phases nonlinear systems," *Inst. Electr. Eng. Proc.—Control Theory Appl.*, vol. 153, no. 4, pp. 379–386, Aug. 2006.
[16] M. Tomizuka, "Dealing with periodic disturbances in controls of mechanical systems," *Annu. Rev. Control*, vol. 32, no. 2, pp. 193–199, Dec. 2008.
[17] W. E. Dixon, E. Zergeroglu, D. M. Dawson, and B. T. Costic, "Repetitive learning control: A Lyapunov-based approach," *IEEE Trans. Syst. Man Cybern. B, Cybern.*, vol. 32, no. 4, pp. 538–545, Aug. 2002.
[18] S. Liuzzo, R. Marino, and P. Tomei, "Adaptive learning control of nonlinear systems by output error feedback," *IEEE Trans. Autom. Control*, vol. 52, no. 7, pp. 1232–1248, Jul. 2007.
[19] F. L. Lewis, A. Yesildirek, and K. Liu, "Multilayer neural-net robot controller with guaranteed tracking performance," *IEEE Trans. Neural Netw.*, vol. 7, no. 2, pp. 388–398, Mar. 1996.
[20] F. L. Lewis, S. Jagannathan, and A. Yesildirek, *Neural Network Control of Robot Manipulators and Nonlinear Systems*. London, U.K.: Tayor & Francis, 1999.

Scaling Up Support Vector Machines Using Nearest Neighbor Condensation

Fabrizio Angiulli and Annabella Astorino

***Abstract*—In this brief, we describe the FCNN-SVM classifier, which combines the support vector machine (SVM) approach and the fast nearest neighbor condensation classification rule (FCNN) in order to make SVMs practical on large collections of data. As a main contribution, it is experimentally shown that, on very large and multidimensional data sets, the FCNN-SVM is one or two orders of magnitude faster than SVM, and that the number of support vectors (SVs) is more than halved with respect to SVM. Thus, a drastic reduction of both training and testing time is achieved by using the FCNN-SVM. This result is obtained at the expense of a little loss of accuracy. The FCNN-SVM is proposed as a viable alternative to the standard SVM in applications where a fast response time is a fundamental requirement.**

***Index Terms*—Classification, large data sets, training-set condensation, nearest neighbor rule, support vector machines (SVMs).**

I. Introduction

The objective of pattern classification is to find a rule, based on external observation, to assign an object to exactly one among several classes. Many algorithms have been indeed devised for automatically distinguishing among different samples on the basis of their patterns. A well-known supervised classification technique is the support vector machine (SVM) algorithm (see [5], [15], [16], and [18]). For many applications, the SVM method, alone or in combination with other methods, yields superior performance with respect to other machine learning options. In general, SVMs work very well in practice, have a small number of tunable parameters in comparison with neural networks, and tend towards global solutions.

Training an SVM is equivalent to solving a convex quadratic programming (QP) problem characterized by a dense, positive–semidefinite matrix with a number of rows equal to the number of training data points. Thus, it is a real challenge to deal with practical applications where the data set is made up of thousands of points. In fact, just computing the matrix for the QP problem is expensive and one may not be able to store it. Moreover, the computing time for solving the QP problem is $O(N^3)$, where N denotes the number of training examples, while the space complexity is $O(N^2)$; these may become prohibitive on large training sets. More in general, applying SVMs on large training sets requires a drastic increase of memory, training time, and prediction time. Indeed, prediction time is proportional to the number of support vectors (SVs), which still increases with the number of data points.

Many algorithms and implementation techniques have been developed for efficiently training SVMs for massive data sets. Among them we mention decomposition techniques and data reduction techniques. Decomposition techniques speed up the SVM training by dividing the original QP problem into smaller pieces, thereby reducing the size of

Manuscript received August 11, 2009; revised November 16, 2009; accepted December 09, 2009. First published January 12, 2010; current version published February 05, 2010.

F. Angiulli is with the Department of Electronic, Computer Science and Systems Engineering, University of Calabria, Rende (CS) 87036, Italy (e-mail: f.angiulli@deis.unical.it).

A. Astorino is with the Institute of High Performance Networking and Computing of the National Research Council, Rende (CS) 87036, Italy (e-mail: astorino@icar.cnr.it).

Color versions of one or more of the figures in this brief are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TNN.2009.2039227

each QP problem. They are among the first proposals to speedup SVMs. Well-known decomposition techniques are the "chunking" algorithm [3], Osuna's decomposition algorithm [13], and the sequential minimal optimization (SMO) [14]. A potential disadvantage of these techniques is that they may need a long training time because they usually require to execute many passages over the data set to reach a reasonable level of convergence. These techniques are at present natively supported by standard SVM tools, as LIBSVM [4].

Different data reduction techniques have been proposed for making the training data of manageable size. The principal idea of this approach is that a subsampling of the training set may speed up a classifier. This can be achieved by randomly removing training points or by selecting some significant samples and ignoring others that can be absorbed, or represented, by those selected. Among the data reduction approaches for SVM proposed in the literature there are the reduced support vector machine (RSVM) [11] and its related variants [8], [10], the cross-training algorithm [2], and the clustering-based SVM (CB-SVM) algorithm [20].

In this brief, we introduce a data reduction method for SVMs, called FCNN-SVM, with the goal of making them scalable to very large data sets. The FCNN-SVM couples the SVM algorithm with the fast nearest neighbor condensation algorithm [1]. Differently from other methods, the FCNN-SVM is based on selection criteria which are guided by the decision boundary rather than by sampling and clustering like techniques, and it can be applied to any kind of data and kernel function.

As a main contribution, it will be experimentally shown that the FCNN-SVM scales well on large and high-dimensional data sets and that it is one or two orders of magnitude faster than the SVM. Moreover, the FCNN-SVM noticeably reduces the size of the model, since, in most cases, the number of SVs is more than halved with respect to the SVM. Thus, a drastic reduction of both training and testing times is achieved by using the FCNN-SVM. As far as test accuracy is concerned, the FCNN-SVM exhibits a little loss of accuracy.

The rest of this work is organized as follows. Section II introduces the FCNN-SVM classifier. Section III experimentally compares the FCNN-SVM and the standard SVM. Finally, Section IV summarizes contributions and concludes the brief.

II. The FCNN-SVM Rule

The FCNN-SVM combines two well-known classification rules, namely, the SVM and the nearest neighbor condensation. As for the definition of SVM and for the nearest neighbor condensation rule the reader is referred to [5], [15], [16], and [18], and to [1], [6], [7], [9], [17], and [19], respectively.

The strong point of SVMs is that they permit to build a classifier which maximizes generalization performances. Nevertheless, their training is costly. The SVs are critical points near the boundary between the two classes, which determine an optimal separating hyperplane. It must be noted that removal of training points that are not SVs has no effect on the hyperplane selection. If one could approximate the right SVs before the training process, then both computing time and memory usage of the SVM would be reduced, and an optimal separating hyperplane could be found by training the SVM on a relatively smaller number of selected points.

From the point of view of the generalization properties, the nearest neighbor rule is not as good as SVM. Nonetheless, a training set which is a consistent subset for the nearest neighbor rule can be found quite efficiently by using the FCNN algorithm [1]. The FCNN method selects a subset of the overall data set having the robust property of correctly classifying the rest of the data set and being mostly composed by points close to the decision boundary. Hence, using the FCNN to reduce the amount of data on which the SVM has to be trained is expected to provide a method with a profitable tradeoff between execution time and prediction accuracy.

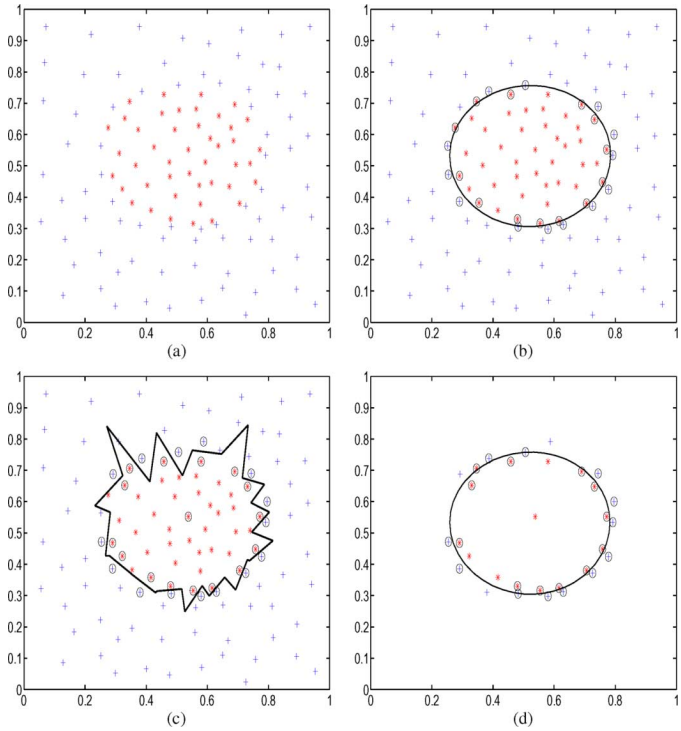

Fig. 1. Example of the FCNN-SVM training.

Given a training set T, the FCNN-SVM classifier is built in two steps.

1) Compute a training set consistent subset S of T for the nearest neighbor rule through the FCNN algorithm.
2) Train an SVM on the consistent subset S.

After selecting a subset of training points having the property of approximating the decision boundary (step 1), a separating hyperplane can be obtained by training an SVM on this subset (step 2).

Fig. 1 shows an example of FCNN-SVM training on a 2-D data set consisting of 121 points in the plane. Fig. 1(a) reports the original training set, composed by two well-separated classes. In Fig. 1(b), the SVs (circled points) are highlighted together with the decision boundary of an SVM trained with parameter $C = 500$ and using a radial basis function (RBF) kernel with $\gamma = 1$. Say SVM^{full} is the above defined classifier (using 26 SVs). Fig. 1(c) shows the FCNN subset (circled points) and the decision boundary of the FCNN rule trained on the original data set. The FCNN subset is composed of 31 points. Note that these points are very close to the decision boundary and most of them coincide with the SVs of the SVM^{full} classifier. Finally, Fig. 1(d) shows the SVs (the 24 circled points) and the decision boundary of the SVM trained on the FCNN subset with parameter $C = 500$ and using an RBF kernel with $\gamma = 1$ that are the SVs and the decision boundary of the FCNN-SVM rule.

Note that while the decision boundary of the FCNN classifier is rather crisp, the decision boundary of the FCNN-SVM classifier is smooth and more accurate. The decision boundary of the FCNN-SVM classifier is practically identical to the decision boundary of the SVM classifier, but it has been obtained at a reduced computational cost, since the training has been accomplished on just the 31 points returned by the FCNN algorithm instead of the 121 points of the overall input training set. Moreover, the number of SVs of the FCNN-SVM classifier is slightly smaller than the number of SV of the standard SVM.

There are different variants of the basic FCNN algorithm. In the following, we make use of the FCNN2 variant (see [1]), which is the fastest one. As far as the temporal cost of the FCNN-SVM method is

TABLE I
EXPERIMENTS ON SMALL DATA SETS

BUPA Liver						
Kernel	Par.	C	SVM		FCNN-SVM	
			Acc.	SVs	Acc.	SVs
Linear		0.1	67.7	240.9	62.8	151.6
		1	72.4	205.4	72.4	139.3
		100	**72.7**	192.7	**73.3**	134.1
RBF	0.01	0.1	58.3	256.0	55.4	153.8
		1	58.3	256.3	55.4	153.9
		100	71.9	199.1	69.2	137.6
	0.1	0.1	58.3	256.5	55.4	153.8
		1	71.0	229.7	67.5	149.9
		100	72.4	178.2	70.1	130.4
	1	0.1	69.8	253.4	56.9	154.6
		1	71.2	198.1	71.8	142.4
		100	66.6	167.8	64.2	130.0
Poly	2	0.1	58.3	256.6	58.3	153.0
		1	67.4	245.0	62.4	153.1
		100	70.9	176.9	69.8	131.1
	3	0.1	58.3	257.4	58.3	153.0
		1	61.8	257.2	58.3	153.2
		100	72.7	188.4	69.2	137.6

Cleveland			
SVM		FCNN-SVM	
Acc.	SVs	Acc.	SVs
84.5	110.2	**84.5**	73.5
85.5	92.8	84.1	61.8
84.5	91.0	82.1	56.9
72.1	152.2	72.1	81.5
85.9	143.4	72.4	81.2
84.5	99.6	83.2	65.4
72.4	154.9	72.4	82.79
84.2	123.1	82.5	80.8
80.1	107.3	71.1	70.2
72.1	231.3	72.1	87.6
76.2	231.0	73.8	87.69
78.2	225.5	76.5	87.4
72.1	155.7	72.1	82.5
79.1	153.3	71.7	82.6
73.7	102.6	71.1	65.8
72.1	158.6	72.1	84.1
78.7	158.6	72.1	84.1
75.4	111.2	67.1	75.1

Pima			
SVM		FCNN-SVM	
Acc.	SVs	Acc.	SVs
77.2	434.5	74.5	298.0
76.7	372.8	76.0	270.4
76.4	359.8	76.4	263.1
65.1	483.0	65.1	298.2
66.4	483.8	65.1	298.9
77.2	367.5	76.7	265.6
65.2	484.2	65.1	298.7
77.3	412.3	76.2	290.0
76.6	347.5	75.9	259.8
75.1	475.3	66.7	300.6
75.6	387.6	76.4	282.0
71.2	347.6	68.1	263.5
65.1	484.9	65.1	299.5
77.7	462.9	70.5	299.5
76.7	349.2	75.5	258.0
65.1	485.1	65.1	300.3
69.1	485.3	65.1	300.6
77.4	366.6	**77.5**	267.7

Ionosphere						
Kernel	Par.	C	SVM		FCNN-SVM	
			Acc.	SVs	Acc.	SVs
Linear		0.1	87.4	122.9	36.6	44.7
		1	87.7	88.8	72.3	41.8
		100	86.6	62.0	76.6	32.3
RBF	0.01	0.1	64.3	227.9	35.7	44.8
		1	89.4	169.8	35.7	45.1
		100	93.1	66.4	84.3	37.7
	0.1	0.1	93.4	223.4	35.7	43.2
		1	93.7	109.4	64.3	43.2
		100	**94.3**	66.3	81.4	34.2
	1	0.1	64.3	240.7	35.7	62.3
		1	92.0	211.8	61.7	62.3
		100	93.1	208.2	70.0	62.3
Poly	2	0.01	64.3	228.1	35.7	48.8
		1	89.1	213.1	35.7	49.3
		100	91.1	83.2	**89.1**	47.0
	3	0.01	64.3	227.8	35.7	53.1
		1	64.6	228.3	35.7	54.0
		100	91.4	126.3	79.4	56.0

Letter			
SVM		FCNN-SVM	
Acc.	SVs	Acc.	SVs
99.1	713.2	95.9	85.6
99.1	507.2	97.3	83.4
99.1	460.7	98.4	75.6
95.9	1365.5	95.9	84.6
99.1	1158.8	95.9	84.2
99.3	445.8	98.8	84.1
99.1	1128.4	95.9	92.4
99.3	560.4	95.9	87.5
99.8	241.6	99.3	86.6
99.4	692.7	95.9	125.4
99.7	385.5	99.2	126.2
99.9	241.4	**99.7**	108.1
95.9	1365.6	95.9	86.4
99.2	872.2	95.9	86.1
99.5	334.3	99.3	86.8
95.9	1366.7	95.9	78.3
98.9	1161.6	95.9	85.9
99.5	377.0	95.9	87.3

Image			
SVM		FCNN-SVM	
Acc.	SVs	Acc.	SVs
96.0	277.8	94.7	139.0
96.3	244.7	95.0	117.2
96.1	210.6	94.7	109.5
53.3	1793.5	14.3	205.2
86.7	1336.6	19.0	203.4
94.6	444.5	92.5	178.8
86.7	1392.9	15.6	207.5
92.5	783.7	45.7	204.3
96.7	305.7	**95.0**	161.6
92.1	1021.4	16.0	208.3
95.6	560.8	74.8	202.3
96.9	406.7	91.2	181.0
96.4	192.9	94.0	121.9
96.5	182.4	92.0	116.8
96.0	174.7	91.6	112.1
95.0	187.1	92.3	122.8
95.4	184.3	93.3	121.3
96.1	183.8	92.9	125.3

TABLE II
FURTHER EXPERIMENTS ON SMALL DATA SETS

Data set	Kernel Type	Kernel Param.	C	SVM		FCNN-SVM	
				Acc.	SVs	Acc.	SVs
BUPA Liver	Linear	–	10	73.3	194.3	72.7	134.7
Cleveland	Linear	–	0.7	85.9	94.2	85.2	62.2
Pima	Linear	–	70	77.6	360.4	77.1	260.4
Ionosphere	RBF	0.08	9	95.1	75.6	92.0	45.7
Letter	RBF	1	10	99.9	252.8	99.7	113.5
Image	RBF	0.1	100	96.7	306.0	95.0	162.0
Average				88.1	213.9	87.0	129.8

concerned, let N be the number of examples in the input training set T, and let n be the size of the training set consistent subset S, then the FCNN algorithm has cost $O(Nn)$.[1] Since the cost of the SVM is cubic in the size of the input training set, the temporal cost of the FCNN-SVM is $O(Nn+n^3)$, which is at most quadratic in the input training set size and cubic in the consistent subset size. The value n depends on the characteristics of the data set, but usually it holds that $n \ll N$, with n corresponding to a few percent of the whole data set size N [1].

[1]The spatial cost of the FCNN algorithm is also $O(Nn)$.

III. EXPERIMENTAL RESULTS

In this section, several experiments involving the FCNN-SVM are described. The performance of the FCNN-SVM rule[2] has been compared with that of LIBSVM [4].

Experiments are organized as follows. We present first the experiments on small data sets, in order to compare data reduction and classification accuracy (Section III-A). Of course, the advantages in terms of temporal cost of the FCNN-SVM rule can be appreciated only when very large data sets have to be processed, which are accounted for in Section III-B. Finally, in Section III-C, we consider some families of synthetical data sets in order to point out both advantageous and critical scenarios for the FCNN-SVM method.

A. Small Data Sets

We applied both FCNN-SVM and SVM on some test problems drawn from the classification literature that are BUPA Liver, Cleveland Heart, Pima Indians, Ionosphere, Letter Recognition, and Image Segmentation, all from the University of California at Irvine (UCI) Machine Learning Repository [12].

[2]The Euclidean distance was employed by the FCNN as distance measure.

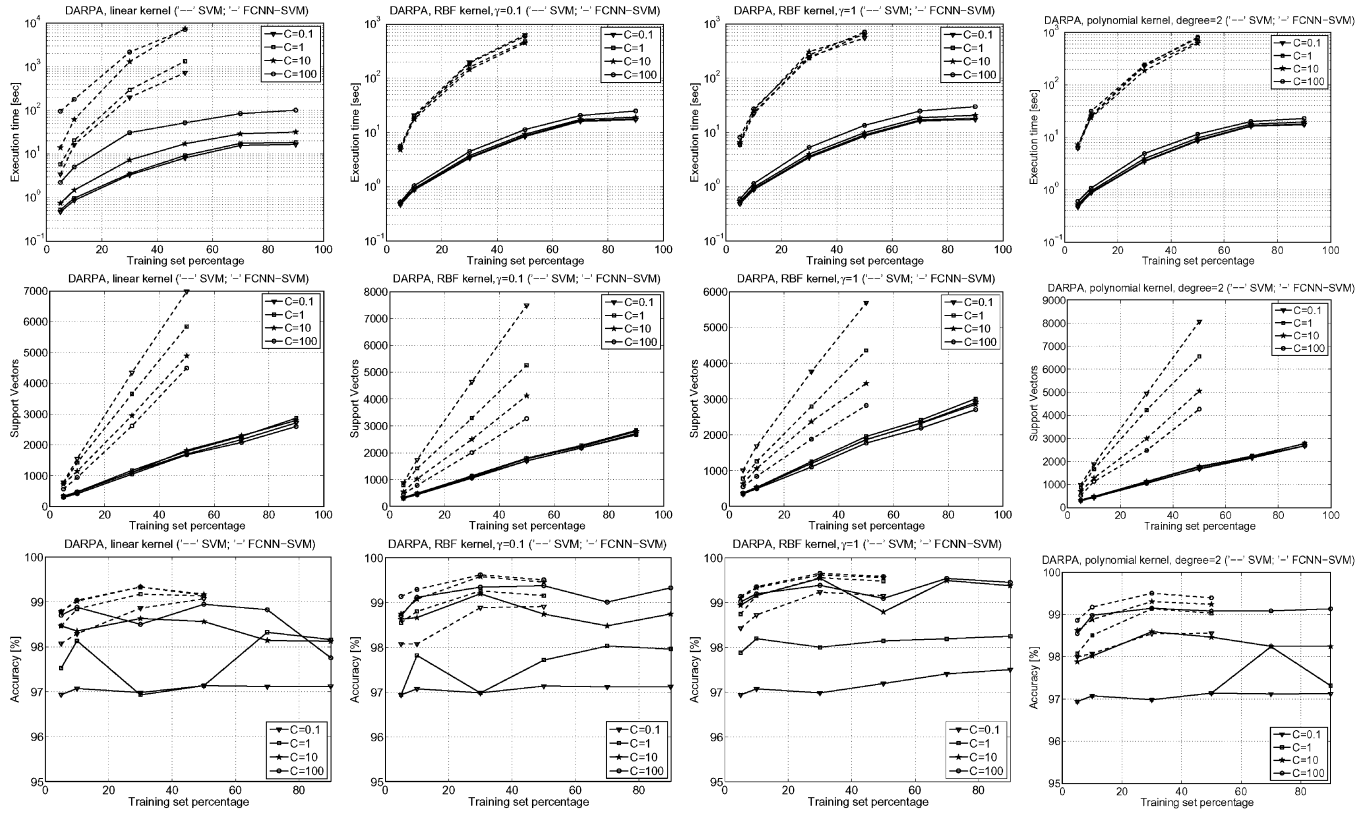

Fig. 2. Experiments on the DARPA 1998 data set.

To make the choice of the free parameters, we considered the following combinations of values on a finite grid:

Penalty C	0.1 1 10 100
Kernel Type	Linear RBF Poly
RBF-parameter	0.01 0.1 1
Poly-parameter	2 3

Classification accuracy (Acc.), measured through tenfold cross validation, and number of SVs are reported in Table I (values for $C = 10$ are not reported in the table due to space limitations). It can be observed that the optimal parameters for the SVM may in general be different from the optimal ones for the FCNN-SVM.

Moreover, for each data set, we have considered a thinner grid of parameter values in the intervals where the SVM method showed a regularly near-optimal behavior in terms of testing classification accuracy and number of SVs; Table II shows the results of these experiments.

As far as the classification accuracy is concerned, the FCNN-SVM exhibits a loss of accuracy with respect to the SVM, and this can be explained as a consequence of the great reduction of number of SVs. Table I shows that the FCNN-SVM is more sensitive to the selection of parameters, since its classification accuracy presents a greater variability with respect to that of the SVM. However, if we select the optimal values of accuracy, either the accuracies of the two methods are comparable or the FCNN-SVM presents a small loss.

B. Scaling Analysis on Very Large Data Sets

In this section, the scaling analysis of the FCNN-SVM and of the SVM on three very large data sets,[3] that are DARPA 1998, Forest Cover, and Checkerboard, is accomplished.

DARPA 1998 is obtained from the Defense Advanced Research Projects Agency 1998 intrusion detection evaluation data[4] and it is composed of 458 301 real vectors each having 23 features, one of which stating whether the vector is associated with a network attack. Forest Cover[5] consists of 581 012 instances each having 54 attributes. Instances are partitioned into seven classes. Checkerboard is a synthetically generated data set composed of 1 000 000 points into the unit square. A 4×4 checkerboard partitions the points into two classes associated with white and black cells of the board.

For each data set, we extracted six random samples of increasing size to form six training sets. Among the remaining objects, we also picked a random sample to form the associated test set.

In particular, on the DARPA 1998 and Forest Cover data sets we considered the same grid of parameters used for the small data sets (Figs. 2 and 3 report some of these experiments; the missing ones showed a very similar behavior). Since the SVM was very slow on these two data sets, it was not executed on the largest samples. On the Checkerboard, we considered only one single combination, which is the RBF kernel with $\gamma = 100$ and C set to 500, since by using these parameters, the SVM performs remarkably well in terms of classification accuracy.

Fig. 2 shows the results of the experiments on the DARPA 1998 data set. The first row of the figure reports the execution times. As an example, when the linear kernel with $C = 10$ was used on half of the data set, the FCNN-SVM terminated in about 17 s, while the SVM employed about 7500 s (a difference of more than two orders of magnitude). Likewise, for all the other kernels, the difference of execution time remained of about the same order. The second row reports the SVs. We observe that the reduction achieved by the FCNN-SVM is noticeable. In almost all cases the number of SVs of the FCNN-SVM is less

[3]Experiments were performed on an Intel Core 2 Duo (single core) 1.83-GHz-based machine with 1 GB of main memory and the Windows operating system.

[4]http://www.ll.mit.edu/IST/ideval/index.html

[5]http://kdd.ics.uci.edu/databases/covertype/covertype.html

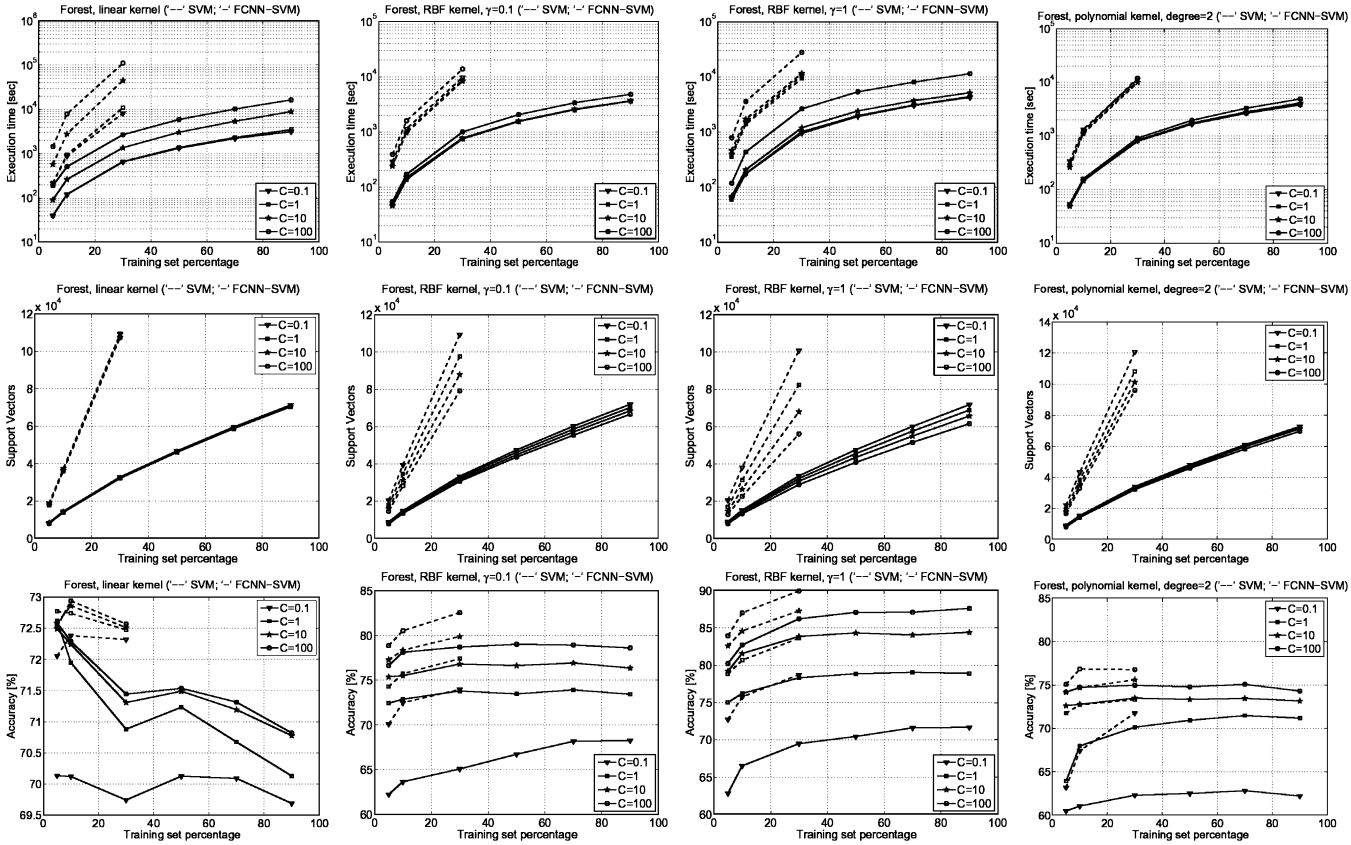

Fig. 3. Experiments on the Forest Cover data set.

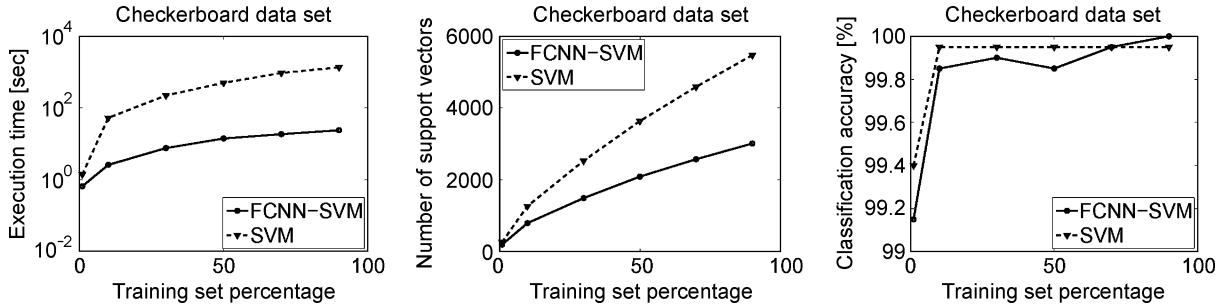

Fig. 4. Experiments on the *Checkerboard* data set.

than half that of the SVM; e.g., by using a polynomial kernel of degree 2 and $C = 0.1$, the number of SVs of the FCNN-SVM is about 20% of that of the SVM. The third row shows the classification accuracy. It is clear that for small values of the parameter C, the FCNN-SVM presents a perceptible loss of accuracy, which, in any case, is within 2%. Interestingly, for greater values of C (e.g., $C = 100$) the accuracies of FCNN-SVM and SVM are comparable.

Fig. 3 shows the results of the experiments on the Forest Cover data set. For this data set, considerations similar to those drawn for the DARPA 1998 data set hold. For example, as for the execution time, by using the linear kernel with $C = 100$ on the 30% sample, the FCNN-SVM terminated in about 2700 s, while the SVM required about 111 000 s.

Fig. 4 shows the results of the experiments on the Checkerboard data set.

While the SVM required about 1350 s, the FCNN-SVM terminated after about 24 s. The FCNN-SVM in this case is about 60 times faster than the SVM. Interestingly, on the largest sample, the FCNN-SVM computed 3012 SVs, while the SVM computed 5474 SVs (55%), without any loss in test accuracy. Indeed, the FCNN-SVM reached the 100% accuracy, while the SVM stabilized on the 99.95% accuracy.

It is interesting to point out that, as far as the execution time and the number of SVs are concerned, as the data set size increases, the performances of the FCNN-SVM improve with respect to those of the SVM. Consider the SVM *relative execution time* (also called *speedup* of the FCNN-SVM) that is the ratio $t_{\mathrm{SVM}}/t_{\mathrm{FCNN-SVM}}$ between the execution time t_{SVM} of the SVM and the execution time $t_{\mathrm{FCNN-SVM}}$ of the FCNN-SVM. We note that, in all the experiments, this ratio increases with the data set size. As an example, the following table reports some SVM relative execution times:

	1%	5%	10%	30%	50%	90%
DARPA	6.91	43.18	35.79	70.51	137.66	–
Forest	2.37	7.68	15.48	41.31	–	–
Checkerboard	2.14	–	20.39	29.26	35.57	56.65

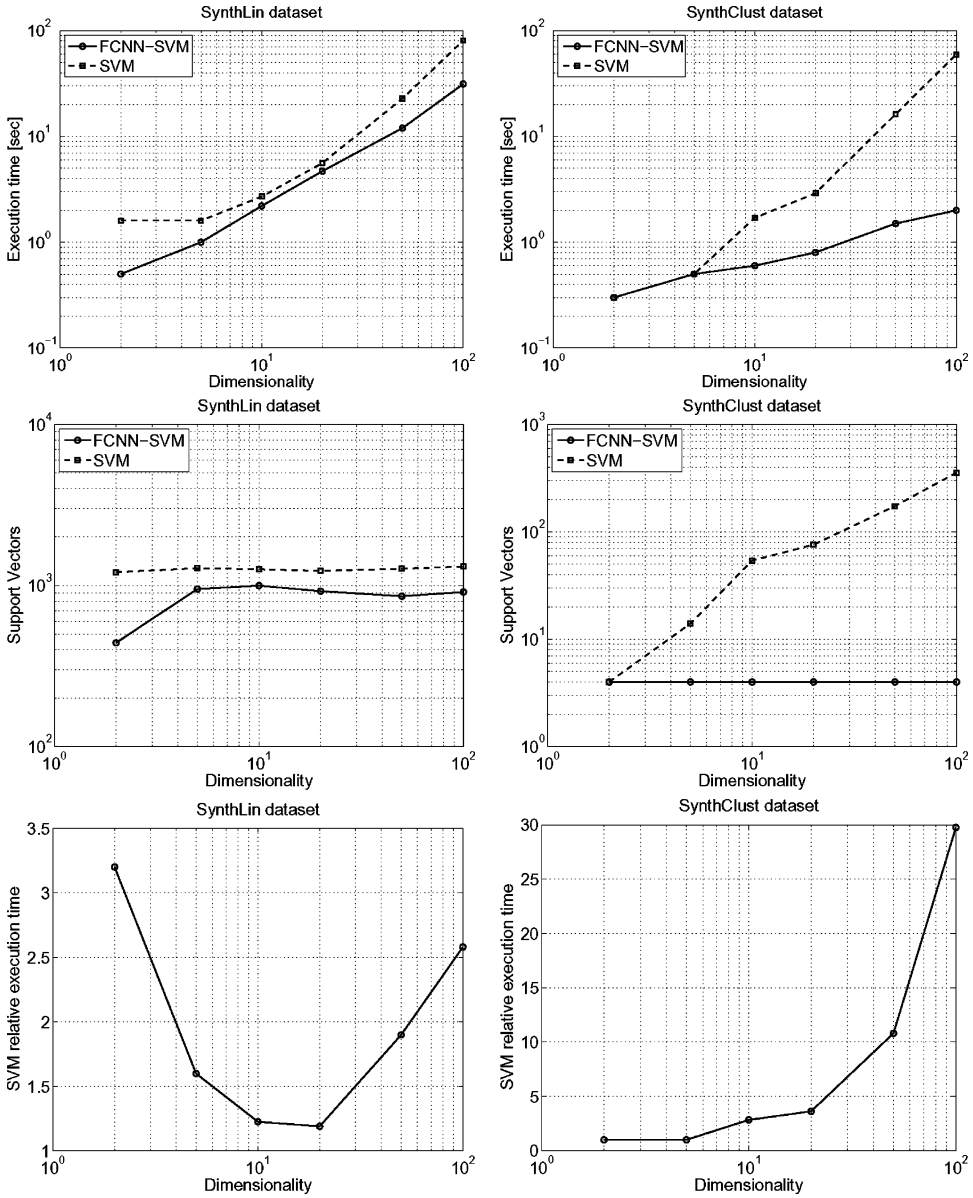

Fig. 5. Experiments on synthetic data sets.

For DARPA 1998 and Forest Cover, the parameters considered in the table above are the linear kernel and C equals 100. From this table, we expect that the larger the data set, the greater the SVM relative execution time. We point out that the above behavior is common to all the other combinations of parameters. Moreover, the FCNN-SVM relative number of SVs% decreases with the data set size. As an example, the following table reports the FCNN-SVM relative number of SVs on the same combination of the parameters above considered:

	1%	5%	10%	30%	50%	90%
DARPA	59%	52%	44%	40%	37%	–
Forest	60%	45%	39%	30%	–	–
Checkerboard	74%	–	63%	59%	57%	55%

Also in this case, the above behavior is common to all the other combinations of parameters.

C. Syntethic Data Sets

Experiments presented in this section are designed to model both advantageous and critical scenarios for the FCNN-SVM method. We considered two families, called *SynthLin* and *SynthClust*, of synthetically generated data sets. The two families differentiate for the data class distribution, while the data sets within the same family differentiate for the dimensionality of the feature space.

The structure of the data set is intentionally simple in order to make immediately intelligible the behavior of the methods. The data sets of the *SynthLin* family are composed of 10 000 points uniformly distributed in the hypercube $[0, 1]^d$, where d is the dimensionality of the feature space ($d \in [2, 100]$). The points are partitioned into two classes by means of a random generated separating hyperplane (classes are almost balanced). Moreover, the labels of 100 randomly selected points (1% of the data set) are flipped in order to simulate the presence of noise. Thus, the classes of these data sets are almost linearly separable, but the points in the data sets "fill" the whole space. The data sets of the *SynthClust* family are composed of 100 000 points distributed in four

well-separated Gaussian clusters in the hypercube $[0, 1]^d$. The clusters are partitioned into two classes in a way that makes them not linearly separable.

Fig. 5 shows the experimental results on the two aforementioned families. As for the *SynthLin* data sets, we used the linear kernel and set the parameter C to 10, since this value guaranteed accuracy close to 99%. As for the *SynthClust* data sets (second row of the figure), we used an RBF kernel with $\gamma = 10$ and set C to 10, since these values guaranteed accuracy close to 100%.

The *SynthLin* data set (first column of Fig. 5) represents a scenario for which the SVM is very appropriate, since classes are almost linearly separable. As for the FCNN-SVM, it must be said that on this kind of data, it does not provide appreciable advantages. Indeed, since the points are uniformly distributed in the unit hypercube and since the class boundary is represented by a hyperplane, as the dimensionality of the space increases, the number of points of one class "facing" the opposite class increases (note the total number of points is held fixed with the dimensionality). Since these are the points contributing to form the FCNN subset, the size of this subset increases as well. As a consequence, the execution time of the FCNN-SVM is comparable to that of the SVM. Moreover, the FCNN-SVM slightly reduces the number of SVs.

The *SynthClust* data set (second column of Fig. 5) represents a common scenario in real-life data, where multidimensional objects are clustered in homogeneous subpopulations. It represents a scenario in which using the FCNN-SVM is very profitable. In fact, it can be observed that in this case the SVM relative execution time worsens as the dimensionality increases, and that the number of SVs selected by the SVM is far greater than that selected by the FCNN-SVM. This can be explained, since the FCNN-SVM works only on the reduced subset selected by the FCNN.

Summarizing, experiments presented in this section serve the purpose of discussing two prototypical scenarios in which using the FCNN-SVM either does not pay much or is very profitable. In general, we can say that the FCNN-SVM does not exhibit large time improvements over the SVM when the number of points contributing to form the FCNN decision boundary is comparable to the size of the training set. This situation could happen, but it is not the standard. In fact, multidimensional real-life data sets are often clustered or even present low intrinsic dimensionality.

Finally, the above experiments show also the scalability behavior of the method with respect to the data dimensionality (while those reported in the preceding section concern scalability with respect to the data set size). Clearly, the absolute execution time should increase with the dimensionality, due to the major cost to compute the distance function. Nonetheless, this does not mean that the speedup of the FCNN-SVM has to worsen. As a matter of fact, for one of the two families shown above (*SynthClust*), the speedup is increasing.

IV. Conclusion

In this brief, we introduced the FCNN-SVM classifier, which greatly reduces the training time and the number of SVs with only a little loss of accuracy with respect to the standard SVM.

References

[1] F. Angiulli, "Fast nearest neighbor condensation for large data sets classification," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 11, pp. 1450–1464, Nov. 2007.

[2] G. H. Bakur, L. Bottou, and J. Weston, "Breaking SVM complexity with cross-training," in *Advances in Neural Information Processing Systems (NIPS)*. Cambridge, MA: MIT Press, 2004.

[3] B. Boser, I. Guyon, and V. Vapnik, "A training algorithm for optimal margin classifiers," in *Proc. 5th Annu. Workshop Comput. Learn. Theory*, 1992, pp. 144–152.

[4] C.-C. Chang and C.-J. Lin, LIBSVM: A Library for Support Vector Machines, 2001 [Online]. Available: http://www.csie.ntu.edu.tw/cjlin/libsvm

[5] N. Cristianini and J. Shawe-Taylor, *An Introduction to Support Vector Machines and Other Kernel-Based Learning Methods*. Cambridge, U.K.: Cambridge Univ. Press, 2000.

[6] W. Gates, "The reduced nearest neighbor rule," *IEEE Trans. Inf. Theory*, vol. IT-18, no. 3, pp. 431–433, May 1972.

[7] P. E. Hart, "The condensed nearest neighbor rule," *IEEE Trans. Inf. Theory*, vol. IT-14, no. 3, pp. 515–516, May 1968.

[8] L.-R. Jen and Y.-J. Lee, "Clustering model selection for reduced support vector machines," in *Proc. Int. Conf. Intell. Data Eng. Autom. Learn.*, 2004, pp. 714–719.

[9] B. Karaçali and H. Krim, "Fast minimization of structural risk by nearest neighbor rule," *IEEE Trans. Neural Netw.*, vol. 14, no. 1, pp. 127–134, Jan. 2002.

[10] Y.-J. Lee, H.-Y. Lo, and S.-Y. Huang, "Incremental reduced support vector machine," in *Proc. Int. Conf. Inf. Cybern. Syst.*, 2003.

[11] Y.-J. Lee and O. L. Mangasarian, "RSVM: Reduced support vector machines," in *Proc. 1st SIAM Int. Conf. Data Mining*, 2001.

[12] P. M. Murphy and D. W. Aha, UCI Repository of Machine Learning Databases, Univ. California Irvine, Irvine, CA, 1992 [Online]. Available: www.ics.uci.edu/~mlearn/MLRepository.html

[13] E. Osuna, R. Freund, and F. Girosi, "An improved training algorithm for support vector machines," in *Proc. IEEE Workshop Neural Netw. Signal Process.*, 1997, pp. 276–285.

[14] J. Platt, "Sequential minimal optimization: A fast algorithm for training support vector machines," in *Advances in Kernel Methods—Support Vector Learning*, B. Schölkopf, C. Burges, and A. Smola, Eds. Cambridge, MA: MIT Press, 1999.

[15] B. Schölkopf, "Advances in kernel methods: Support vector learning," in *Advances in Neural Information Processing System*, C. J. C. Burges and A. J. Smola, Eds. Cambridge, MA: MIT Press, 1999.

[16] J. Shawe-Taylor and N. Cristianini, *Kernel Methods for Pattern Analysis*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[17] G. Toussaint, "Proximity graphs for nearest neighbor decision rules: Recent progress," in *Proc. Symp. Comput. Statist.*, Montreal, QC, Canada, 2002, pp. 17–20.

[18] V. Vapnik, *The Nature of the Statistical Learning Theory*. New York: Springer-Verlag, 1995.

[19] D. R. Wilson and T. R. Martinez, "Reduction techniques for instance-based learning algorithms," *Mach. Learn.*, vol. 38, no. 3, pp. 257–286, 2000.

[20] H. Yu, J. Yang, and J. Han, "Classifying large data sets using SVMs with hierarchical clusters," in *Proc. Int. Conf. Knowl. Disc. Data Mining*, 2003, pp. 306–315.

Fast Nearest Neighbor Condensation for Large Data Sets Classification

Fabrizio Angiulli

Abstract—This work has two main objectives, namely, to introduce a novel algorithm, called the Fast Condensed Nearest Neighbor (FCNN) rule, for computing a training-set-consistent subset for the nearest neighbor decision rule and to show that condensation algorithms for the nearest neighbor rule can be applied to huge collections of data. The FCNN rule has some interesting properties: it is order independent, its worst-case time complexity is quadratic but often with a small constant prefactor, and it is likely to select points very close to the decision boundary. Furthermore, its structure allows for the triangle inequality to be effectively exploited to reduce the computational effort. The FCNN rule outperformed even here-enhanced variants of existing competence preservation methods both in terms of learning speed and learning scaling behavior and, often, in terms of the size of the model while it guaranteed the same prediction accuracy. Furthermore, it was three orders of magnitude faster than hybrid instance-based learning algorithms on the MNIST and Massachusetts Institute of Technology (MIT) Face databases and computed a model of accuracy comparable to that of methods incorporating a noise-filtering pass.

Index Terms—Classification, large and high-dimensional data, nearest neighbor rule, prototype selection algorithms, training-set-consistent subset.

1 Introduction

The nearest neighbor (NN) decision rule [10] assigns to an unclassified sample point the classification of the nearest of a set of previously classified points. For this decision rule, no explicit knowledge of the underlying distributions of the data is needed. A strong point of the NN rule is that for all distributions, its probability of error is bounded above by twice the Bayes probability of error [10], [27], [16]. That is, it may be said that half of the classification information in an infinite size sample set is contained in the NN.

The naive implementation of the NN rule requires the storage of all of the previously classified data and then the comparison of each sample point to be classified to each stored point. In order to reduce both space and time requirements, several techniques to reduce the size of the stored data for the NN rule have been proposed (see [32] and [28] for a survey), referred to as training-set reduction, training-set condensation, reference set thinning, and prototype selection algorithms. In particular, among these techniques, *training-set-consistent* ones aim to select a subset of the training set that classifies the remaining data correctly through the NN rule.

Using a training-set-consistent subset, instead of the entire training set, to implement the NN rule has the additional advantage that it may guarantee better classification accuracy. Indeed, Karaçali and Krim [22] showed that the Vapnik-Chervonenkis (VC) dimension of an NN classifier is given by the number of reference points in the training set. Thus, in order to achieve a classification rule with controlled generalization, it is better to replace the training set with a small consistent subset. Unfortunately, computing a minimum-cardinality training-set-consistent subset for the NN rule turns out to be intractable [30].

Several training-set-consistent condensation algorithms have been introduced in the literature. We point out that among the criteria characterizing condensation methods, learning speed is usually neglected. However, in order to manage huge amounts of data, methods exhibiting good scaling behavior are definitively needed. This work introduces a novel order-independent algorithm for finding a training-set-consistent subset for the NN rule, called the Fast Condensed NN (FCNN) rule, shows that it is scalable on large multidimensional training sets, and compares it with existing condensation methods.

The rest of the paper is organized as follows. In Section 2, previous approaches are described and compared to the approach here proposed, and the contribution of this work is illustrated. In Section 3, the FCNN rule is described, and its main properties are stated. In Section 4, experimental results are presented together with a thorough comparison with existing methods. Finally, in Section 5, the strengths and weaknesses of the different condensation methods are discussed, and conclusions are drawn.

2 Related Works and Contribution

Starting from the seminal work in [21], several training-set condensation algorithms have been introduced in the literature, also known as instance-based [2], lazy [1], memory-based [26], and case-based learners [29]. These methods can be grouped into three main categories depending on the objectives that they want to achieve [8], that is, *competence preservation*, *competence enhancement*, and *hybrid approaches*.

• *The author is with the Dipartimento di Elettronica, Informatica e Sistemistica, Università della Calabria, Via P. Bucci, 41C, 87036 Rende (CS), Italy. E-mail: f.angiulli@deis.unical.it.*

Manuscript received 20 Sept. 2006; revised 30 June 2007; accepted 11 July 2007; published online 1 Aug. 2007.
For information on obtaining reprints of this article, please send e-mail to: tkde@computer.org, and reference IEEECS Log Number TKDE-0443-0906.
Digital Object Identifier no. 10.1109/TKDE.2007.190645.

The goal of competence preservation methods is to compute a training-set-consistent subset, removing superfluous instances that will not affect the classification accuracy of the training set. Competence enhancement methods aim to remove noisy instances in order to increase classifier accuracy. Finally, hybrid methods search for a small subset of the training set that simultaneously achieves both noisy and superfluous instances elimination.

Competence enhancement and preservation methods can be combined in order to achieve the same objectives as hybrid methods. However, although the goals of enhancement and preservation methods are clearly separated, that is, smoothing the decision boundary in the former case and computing a possibly small training-set-consistent subset in the latter, often it is not clear how subsets computed by hybrid methods can be related to the sets computed by the two other methods. Thus, searching for a small training-set-consistent subset is a relevant task *per se*, improving both classification speed and classifier accuracy [22], and computationally hard [30].

Next, we first describe training-set-consistent subset methods for the NN rule and some other related work and then point out the contributions of this work. A detailed survey of condensation methods can be found in [32] and [28].

2.1.1 Condensed Nearest Neighbor (CNN) Rule

The concept of a training-set-consistent subset was introduced by Hart [21] together with an algorithm, called the CNN rule, to determine a consistent subset of the original sample set.

The algorithm uses two bins, called S and T. Initially, the first sample of the training set is placed in S, whereas the remaining samples of the training set are placed in T. Then, one pass through T is performed. During the scan, whenever a point of T is misclassified using the content of S, it is transferred from T to S. The algorithm terminates when no point is transferred during a complete pass of T.

The motivation for this heuristic is that misclassified data lie close to the decision boundary. Nevertheless, the CNN rule may select points far from the decision boundary. Furthermore, the CNN rule is order dependent, that is, it has the undesirable property that the consistent subset depends on the order in which the data is processed. Thus, multiple runs of the method over randomly permuted versions of the original training set should be executed in order to settle the quality of its output [3].

2.1.2 Modified Condensed Nearest Neighbor (MCNN) Rule

The MCNN rule [14] computes a training-set-consistent subset in an incremental manner.

The consistent subset S is initially set to the empty set. During each main iteration of the algorithm, first, the points S_t of the training set T misclassified by using S are selected. Then, the set of centroids[1] C of the points in S_t are computed and used to classify S_t. S_t is thus partitioned into two sets, S_r and S_s, of points that are correctly classified and misclassified, respectively, by using the centroids C. If set S_s is empty, then the set S is augmented with the set C; otherwise, S_t is set to S_r, and the process is repeated until S_s becomes empty. The algorithm terminates when there are no misclassified points of T by using S.

Different from the CNN rule, the MCNN rule is order independent, that is, it always returns the same consistent subset independent of the order in which the data is processed. As claimed by the authors of the algorithm, the MCNN rule may work well for data with a Gaussian distribution or that can be split up into regions with a Gaussian distribution, but, in general, it is unlikely to select points close to the decision boundary. Also, during each iteration of the MCNN rule, at most m points can be added to the subset S, where m is the number of classes in T; thus, the method might require a lot of iterations to converge.

2.1.3 Structural Risk Minimization Nearest Neighbor Rule

In order to compute a small consistent subset S of the training set T, Karaçali and Krim [22] proposed the following algorithm, called Structural Risk Minimization using the NN rule (NNSRM).

Assume that the training set T contains only two class labels.[2] First, all pairwise distances among points having different class labels are computed. Let $\{x_i, y_i\}$ denote the pair having the ith smallest distance, $i \geq 1$. The set S is initialized to $\{x_1, y_1\}$, that is, to the closest points x_1 and y_1 from the two classes, and a counter k is initialized to 2. Next, until set S misclassifies at least a point in T, the following steps are performed: If $\{x_k, y_k\}$ is not contained in S, then S is augmented with both x_k and y_k; in any case, k is set to $k+1$. That is, the algorithm performs one pass on the pairs of points from the two classes ordered in increasing distance, and until the initially empty set S misclassifies the training set T, whenever a pair is not contained in S, it is added to S.

The intuition underlying the method is that since the nearest pairs of points from the opposite classes lie in the regions where the domains of the two classes are closest, that is, where most classification errors occur, they must be considered first. However, the method is costly. Indeed, the complexity of the NNSRM algorithm is $O(|T|^3)$. Furthermore, we note that the size of the condensed set computed is sensitive to the pairs having the greatest distances. Indeed, assume that two points from opposite classes in the training set are far from the other training-set points and such that their distance is greater than the distance from any other pair of the remaining points from the opposite classes. Then, a training-set-consistent subset must contain these two points and, hence, all the training-set points.

2.1.4 Reduced Nearest Neighbor (RNN) Rule

The RNN rule [20] is a postprocessing step that can be applied to any other competence preservation method. It works as follows: First, subset S is set equal to training set T or to a training-set-consistent subset of T. Then, the points of S whose deletion from S does not leave any point of T misclassified are eliminated from S. Experiments have

1. Given a set S of points having the same class label, the *centroid* c of S is the point c of S that is closest to the geometrical center of S. The set of centroids C of a set of points $S = S_1 \cup \ldots \cup S_n$, where each $S_i (1 \leq i \leq n)$ is a maximal subset of points of S having the same class label, is the set $C = \{c_1, \ldots, c_n\}$, where c_i is the centroid of $S_i (1 \leq i \leq n)$.

2. They also provided a similar algorithm working on data with more than two class labels.

shown that this rule yields a slightly smaller subset than the CNN rule, but it is costly.

2.1.5 Minimum-Cardinality Methods

Methods previously discussed compute a training-set-consistent subset in an incremental or decremental manner and have polynomial execution time requirements. Next, competence preservation methods whose aim is to compute a minimum-cardinality training-set-consistent subset (an NP-hard task, see [30]) are briefly described.

The Selective NN (SNN) rule [25] computes the smallest training-set-consistent subset S of T having the following additional property: Each point of T is closer to a point of S of the same class than to any point of T of a different class. This set is also called a *selective subset*. This algorithm runs in exponential time (the problem of computing the minimum-cardinality selective subset has been proved to be NP-hard [30]) and, hence, it is not suitable on large training sets.

The Minimal Consistent Subset (MCS) rule [12] aims to compute a minimum-cardinality training-set-consistent subset. The algorithm, based on the computation of the so-called nearest unlike neighbors [13], is quite complex. Furthermore, counterexamples have been found to the conjecture that it computes a minimum-cardinality subset.

Approximate optimization methods such as tabu search, gradient descent, evolutionary learning, and others have been used to compute subsets close to the minimum-cardinality one. These algorithms can be applied in a reasonable amount of time only to a small- or medium-sized data set. The work in [24] provides a comparison of a number of these techniques.

2.1.6 Other Approaches

Finally, we mention the literature less related to the competence preservation task but involving complementary or alternative approaches, concerning *decision-boundary-consistent subset* methods [7], which compute a subset of the original training set preserving the decision boundary, condensing and editing techniques through the use of *proximity graphs* [28], *NN search* techniques [11], [19], which can alleviate the cost of searching for the NN of a query point at classification time, and *competence enhancement* and *hybrid* methods [32], introduced early in this section.

If the training set is clean, then using a consistent method is very appropriate. In the presence of noise, if the accuracy of the consistent methods is not adequate, they can be combined with competence enhancement methods (for example, Wilson editing or multiedit [31], [15]). Alternatively, accuracy may be improved by using the k-NN rule [18], [17], the generalization of the NN rule in which a new object is assigned to the class with the most members present among its k NNs in the training set. Indeed, the probability of error of the k-NN rule asymptotically approaches the Bayes error. The accuracy of CNN classifiers can be also enhanced by combining *multiple classifiers*, as done in [3], where it is proposed to train multiple CNN classifiers on smaller training sets and to take a vote over them, or in [5], [6], where the multiple feature subsets (MFS) algorithm is described, combining multiple NN classifiers, each using only a random subset of the features.

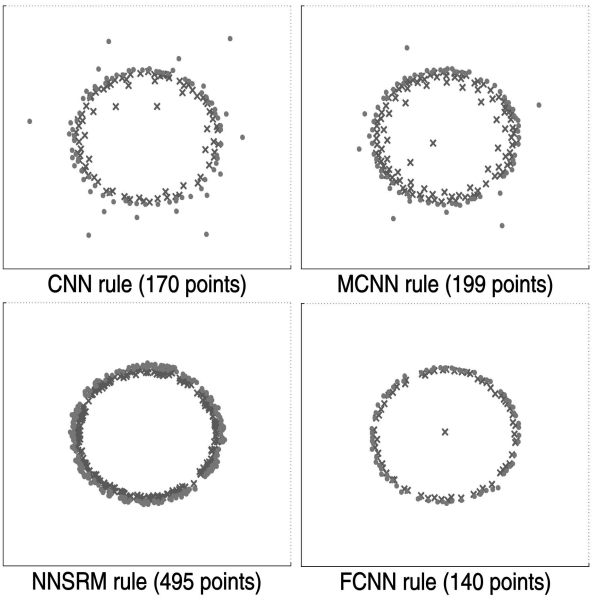

Fig. 1. Example of training-set-consistent subsets computed by the CNN, MCNN, NNSRM, and FCNN rules.

2.2 Contribution

This work introduces a novel algorithm for the computation of a training-set-consistent subset for the NN rule.[3] The algorithm, called the FCNN rule, works as follows.

First, the consistent subset S is initialized to the centroids of the classes contained in the training set T. Then, during each iteration of the algorithm, for each point p in S, a point q of T belonging to the Voronoi cell of p[4] but having a different class label is selected and added to S. The algorithm stops when no further point can be added to S, that is, when T is correctly classified using S.

Despite being quite simple, the FCNN rule has some desirable properties. Indeed, it is order independent, its worst-case time complexity scales as the product of the cardinality of the training set and of the condensed set, it requires few iterations to converge, and it is likely to select points very close to the decision boundary.

For example, Fig. 1 compares the consistent subsets computed by the CNN, MCNN, NNSRM, and FCNN rules on a training set composed of 9,000 points uniformly distributed into the unit square and partitioned into two classes by a circle of diameter 0.5.

As already noted elsewhere [32], training-set reduction algorithms can be characterized by their storage reduction, classification speed increase, generalization accuracy, noise tolerance, and learning speed. Among these criteria, learning speed is usually neglected. However, in order to be practicable on large training sets or in knowledge discovery applications requiring a learning step in their cycle, the method should exhibit good learning behavior.

The contribution of this work can be summarized as follows:

3. A preliminary version of this work appeared in [4].

4. The Voronoi cell of point $p \in S$ is the set of all points that are closer to p than to any other point in S.

- A novel order-independent algorithm, called the FCNN rule, for the computation of a training-set-consistent subset for the NN rule is presented.
- It is proved that the worst-case time complexity of FCNN is quadratic with an often small constant prefactor that corresponds to the reduction ratio. Furthermore, an implementation cleverly exploiting the triangle inequality and sensibly reducing this prefactor is described.
- It is shown that the FCNN method scales well on large-sized multidimensional data sets.
- The FCNN rule is compared with both standard and here-enhanced implementations of existing competence preservation algorithms, showing that it outperforms the other methods in terms of learning speed and learning scaling behavior and, often, in terms of size of the model.
- The FCNN rule is compared with hybrid instance-based learning algorithms on the MNIST and MIT Face databases showing that it is at least three orders of magnitude faster while guaranteeing a comparable accuracy.
- An extension of the basic rule taking into account the k NNs is presented, and it is shown that the computed subset has the same accuracy as the hybrid methods incorporating a noise-filtering step.

In its entirety, this is the first work providing NN condensation algorithms that have been shown experimentally to be scalable on large multidimensional data sets.

3 The FCNN Rule

We start by giving some preliminary definitions.

In the following, we denote by T a labeled training set from a metric space with distance metrics d.

Let p be an element of T. We denote by $nn(p,T)$ the NN of p in T according to the distance d. If $p \in T$, then p itself is its first NN. We denote by $l(p)$ the label associated with p.

Given a point q, the NN rule $\mathrm{NN}(q,T)$ assigns to q the label of the NN of q in T, that is, $\mathrm{NN}(q,T) = l(nn(q,T))$.

A subset S of T is said to be a *training-set-consistent subset of* T if for each $p \in (T - S)$, $l(p) = \mathrm{NN}(p,S)$.

Let S be a subset of T and let p be an element of S. We denote by $\mathrm{Vor}(p,S,T)$ the set $\{q \in T | p = nn(q,S)\}$, that is, the set of the elements of T that are closer to p than to any other element p' of S. Furthermore, we denote by $\mathrm{Voren}(p,S,T)$ the set of *Voronoi enemies* of p in T with respect to S, defined as $\{q \in \mathrm{Vor}(p,S,T) \mid l(q) \neq l(p)\}$.

We denote by $Centroids(T)$ the set containing the centroids of each class label in T.

The following theorem states the property exploited by the FCNN rule in order to compute a training-set-consistent subset.

Theorem 3.1. *S is a training-set-consistent subset of T for the NN rule if and only if for each element p of S, $\mathrm{Voren}(p,S,T)$ is empty.*

Proof. ($\Rightarrow$) By contradiction, assume that there is an element p of S such that $\mathrm{Voren}(p,S,T)$ is not empty. Then, at least an element q of $\mathrm{Voren}(p,S,T)$ and, hence, of T is such that $\mathrm{NN}(q,S) = l(nn(q,S)) = l(p) \neq l(q)$, and S is not training set consistent.

($\Leftarrow$) First of all, note that $\left(\cup_{p \in S} \mathrm{Vor}(p,S,T)\right) = T$. Thus, for each $q \in T$, there is $p \in S$ such that $q \in \mathrm{Vor}(p,S,T)$, and since $\mathrm{Voren}(p,S,T)$ is empty, it holds that $l(nn(q,S)) = l(p) = l(q)$. □

Algorithm FCNN rule
Input: A training set T;
Output: A training set consistent subset S of T;
Method:
  $S = \emptyset$;
  $\Delta S = Centroids(T)$;
  while $(\Delta S \neq \emptyset)$ {
    $S = S \cup \Delta S$;
    $\Delta S = \emptyset$;
    for each $(p \in S)$
      $\Delta S = \Delta S \cup \{rep(p, \mathrm{Voren}(p,S,T))\}$;
  }
  return(S);

Fig. 2. The FCNN rule.

3.1 Algorithm

The algorithm FCNN rule is shown in Fig. 2. It initializes the consistent subset S with a seed element from each class label of the training set T. In particular, the seeds employed are the centroids of the classes in T.

The algorithm works in an incremental manner: During each iteration, the set S is augmented until the stop condition, given by Theorem 3.1, is reached. For each element of S, a representative element of $\mathrm{Voren}(p,S,T)$ with respect to p, denoted as $rep(p, \mathrm{Voren}(p,S,T))$ in Fig. 2, is selected and inserted into S.

Given $p \in T$ and a subset $X \subseteq T$, the representative $rep(p,X)$ of X with respect to p can be defined in different ways. We investigate the behavior of two different definitions for $rep(p,X)$. The first definition, we call FCNN1 the variant of the FCNN rule using this definition, selects the NN of p in X, that is, $rep(p,X) = nn(p,X)$. The second definition, we call FCNN2 the variant of the FCNN rule using it, selects the class centroid in X closest to p, that is, $rep(p,X) = nn(p, Centroids(X))$.

If during an iteration, no new element can be added to S, then by Theorem 3.1, S is a training-set-consistent subset of T, and the algorithm terminates, returning the set S.

Figs. 3 and 4 report an example of the execution of the FCNN1 and FCNN2 rules on the same data set. The data set considered is composed of 2,000 points on the plane. Half of the points belong to one of two concentric spirals representing two distinct classes. In this case, the FCNN1 rule performs nine iterations and returns a subset composed of 54 points, whereas the FCNN2 rule performs 10 iterations and computes a subset composed of 74 points. It is clear from these figures that the FCNN rule requires few iterations to converge to a solution. Indeed, the number of points contained in the subset could double at the end of each iteration. This is due to the fact that for each point p such that $\mathrm{Voren}(p,S,T)$ is not empty, a new point is added to the set S.

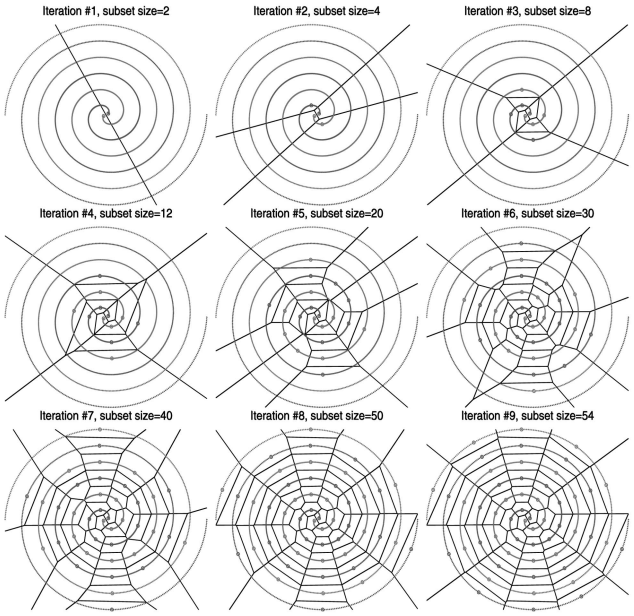

Fig. 3. Example of the execution of the FCNN1 rule.

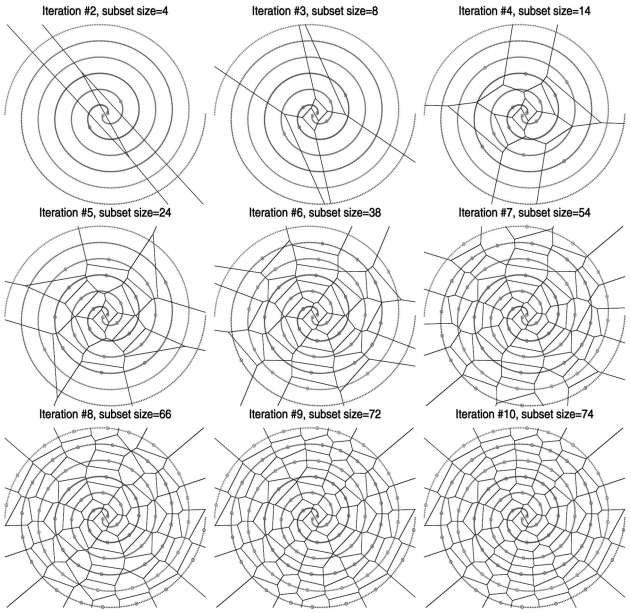

Fig. 4. Example of the execution of the FCNN2 rule.

As discussed above, the FCNN1 and FCNN2 rules build the set ΔS by selecting a representative of the Voronoi enemies of all the points in the current subset S. It is thus of interest to analyze the behavior of the FCNN rule in the special case in which the set ΔS is constrained to be composed of one single point per iteration, that is, in the special case in which ΔS is built by selecting the representative of the Voronoi enemies of only one of the elements of S.

Such an element can be defined in different ways, but a natural choice is to select the point p^* of S such that $|\mathrm{Voren}(p^*, S, T)|$ is maximum, that is, the point having most Voronoi enemies. In the following, we will call the FCNN3 rule the variant of the FCNN rule whose set ΔS is built by selecting only point $nn(p^*, \mathrm{Voren}(p^*, S, T))$ and FCNN4 the variant whose set ΔS is built by selecting only point $nn(p^*, Centroids(\mathrm{Voren}(p^*, S, T)))$. Different from FCNN1 and FCNN2, both FCNN3 and FCNN4 initialize the set S by using only the centroid of the most populated class.

The following theorem states the main properties of the algorithm.

Theorem 3.2. *The algorithm FCNN rule 1) terminates in a finite time, 2) computes a training-set-consistent subset, and 3) is order independent.*

The proof of Theorem 3.2 is reported in the Appendix. Fig. 5 shows the implementation of the FCNN1 rule. For the sake of clarity, the treatment of the ties described in Theorem 3.2 is not reported in the figure.

The algorithm maintains in the array $nearest$ for each training-set point q the closest point $nearest[q]$ of q in the set S and in the array rep for each point p in S its current representative $rep[p]$ of the misclassified points lying in the Voronoi cell of p.

During each iteration, the array $nearest$ and rep must be updated. Let ΔS be the set of points added to the set S at the beginning of the current iteration. To update the array $nearest$, it is sufficient to compare the training-set points in $(T - S)$ with the points in the set ΔS, as the NNs in $S - \Delta S$ of the points in $(T - (S - \Delta S))$ were computed in the previous iterations and are already stored in $nearest$.

After having computed the closest point $nearest[q]$ in ΔS and, hence, in S of a point q in $(T - S)$, the array rep can be updated efficiently as follows: If the label of q is different from the label of $nearest[q]$, then q is misclassified. In this case, if the distance from $nearest[q]$ to q is less than the distance from $nearest[q]$ to its current associated representative $rep[nearest[q]]$, then $rep[nearest[q]]$ is set to q.

The following theorem states an upper bound to the complexity of the FCNN rule.

Algorithm FCNN1 rule
Input: A training set T;
Output: A training set consistent subset S of T;
Method:
for each $(p \in T)$
$nearest[p]$ = undefined;
$S = \emptyset$;
$\Delta S = Centroids(T)$;
while $(\Delta S \neq \emptyset)$ {
$S = S \cup \Delta S$;
for each $(p \in S)$
$rep[p]$ = undefined;
for each $(q \in (T - S))$ {
for each $(p \in \Delta S)$
if $(\mathrm{d}(nearest[q], q) > \mathrm{d}(p, q))$
$nearest[q] = p$;
if $((l(q) \neq l(nearest[q]))$ &&
$(\mathrm{d}(nearest[q], q) < \mathrm{d}(nearest[q], rep[nearest[q]])))$
$rep[nearest[q]] = q$;
}
$\Delta S = \emptyset$;
for each $(p \in S)$
if ($rep[p]$ is defined) $\Delta S = \Delta S \cup \{rep[p]\}$;
}
return(S);

Fig. 5. The FCNN1 rule.

Theorem 3.3. *The FCNN1 rule requires at most $|T| \cdot |S|$ distance computations using $O(|T|)$ space.*

Proof. The algorithm shown in Fig. 5 has space complexity $O(|T|)$, as it employs the two arrays $nearest$ and rep, having size $|T|$ and $|S|$, respectively, with $|S| \leq |T|$.

Let S_{i-1} and ΔS_{i-1} denote, respectively, the values of sets S and ΔS at the beginning of the ith iteration of the algorithm in Fig. 5 and let S_i denote set $S_{i-1} \cup \Delta S_{i-1}$. In order to find the NN among the points in ΔS_{i-1} of each training-set point in $(T - S_i)$, the algorithm computes $(|T| - |S_i|)|\Delta S_{i-1}|$ distances.

Note that in order to avoid repeated distance calculations, distances $\{\mathrm{d}(q, nearest[q]) \mid q \in T\}$ and $\{\mathrm{d}(p, rep[p]) \mid p \in S\}$ can be maintained into two additional arrays of floating-point numbers having size $|T|$ and $|S|$, respectively, without additional asymptotic space requirements.

Thus, the overall number of distances computed is

$$|T| \cdot \sum_i |\Delta S_i| - \sum_i |S_i| \cdot |\Delta S_{i-1}| = |T| \cdot |S| - \sum_{i<j} |\Delta S_i| \cdot |\Delta S_j|$$

and, hence, $|T| \cdot |S|$ is an upper bound to the number of distance computations required. □

FCNN2 has a very similar implementation. The only difference lies in the update of the array rep. Indeed, in order to determine the centroids of the misclassified points of each Voronoi cell induced by the points in S, the FCNN2 rule performs a supplemental training-set scan at the end of each main iteration. Hence, as for the worst-case time complexity of the FCNN2 rule, it requires at most $\sum_i [(|T| - |S_i|)|\Delta S_{i-1}| + (|T| - |S_i|)] \leq |T| \cdot (|S| + t) \leq 2|T| \cdot |S|$ distance computations, where $t \leq |S|$ is the number of iterations required to converge. As we will see in the experimental results section, the number t of iterations performed by the FCNN2 rule is always small (up to 20–30 iterations, even when S is composed of tens of thousands objects).

3.2 Exploiting the Triangle Inequality

In a metric space, the FCNN rule can take advantage of the triangle inequality in order to avoid the comparison of each point in $(T - S)$ with each point in ΔS.

To this aim, the grouping in Voronoi cells of the training-set points is exploited. During the generic ith main iteration, for each $p \in S_{i-1}$, the Voronoi cell $\mathrm{Vor}(p, S_{i-1}, T)$ is visited, and the points $q \in (\mathrm{Vor}(p, S_{i-1}, T) - \{p\})$ are compared with the points in ΔS_{i-1}. Before starting the comparison, the distances between the point p and the points in ΔS_{i-1} are computed, and the points in ΔS_{i-1} are sorted in order of increasing distance from p. Then, instead of comparing each point q in $\mathrm{Vor}(p, S_{i-1}, T)$ with each point in ΔS_{i-1}, each q is compared with the points in ΔS_{i-1} having a distance from p less than twice the distance from q and p. Indeed, by the triangle inequality, they are all and the only points of ΔS_{i-1} candidate to be closer to q than p.

We note that the partitioning of points in the Voronoi cells is induced by the array $nearest$. In order to retrieve the points lying in the same Voronoi cell efficiently, the array $nearest$ does not directly store the identifiers of training-set NNs. Rather, it is used to implement lists of points lying in the same Voronoi cell. To this aim, each entry $nearest[q]$ stores the identifier of the successor of q in the list associated with the Voronoi cell containing q. The identifier of the first point of each list is then stored in an additional array having size $|S_i|$.

This implementation of the FCNN rule requires $\sum_{i<j} |\Delta S_i| \cdot |\Delta S_j|$ additional distance computations and has no additional asymptotic space requirements. Thus, the upper bound of Theorem 3.3 remains unchanged. Furthermore, $\sum_{i<j} |\Delta S_i| \cdot |\Delta S_j| \cdot \log |\Delta S_j|$ worst-case comparisons of floating-point numbers to sort distances are additionally required, but it must be noticed that the former operation is more expensive (its cost being related to the dimensionality of the feature space) than the latter (which has instead a constant cost). Nevertheless, the last implementation guarantees great savings in terms of computed distances and definitively outperforms the previous one, as will be confirmed by the experimental results.

3.3 Extension to k Nearest Neighbors

The NN decision rule can be generalized to the case in which the k NNs are taken into account. In such a case, a new object is assigned to the class with the most members present among the k NNs of the object in the training set. This rule has the additional property that it provides a good estimate of the Bayes error and that its probability of error asymptotically approaches the Bayes error [18], [17]. Let T be a training set and let p be an element of T. We denote by $nn_k(p, T)$ the kth NN of p in T and by $nns_k(p, T)$ the set $\{nn_i(p, T) \mid 1 \leq i \leq k\}$. Given a point q, the k-NN rule $\mathrm{NN}_k(q, T)$ assigns to q the label of the class with the most members present in $nns_k(p, T)$.

The FCNN algorithm can be directly adapted to deal with k NNs by using the following notion of consistency: A subset S of T is said to be a *k-training-set-consistent subset of T* if for each $p \in (T - S)$, $l(p) = \mathrm{NN}_k(p, S)$. Thus, this definition guarantees that the objects in $T - S$ are correctly classified by S through the k-NN rule. For $k = 1$, the above definition coincides with the definition provided in Section 3.

The FCNN rule can directly compute a k-training-set-consistent subset S of T by using the following definition of $\mathrm{Voren}(p, S, T)$. Let p be an element of S. We denote by $\mathrm{Voren}_k(p, S, T)$ the set

$$\{q \in (\mathrm{Vor}(p, S, T) - \{p\}) \mid l(q) \neq \mathrm{NN}_k(q, S)\}.$$

Thus, the set $\mathrm{Voren}_k(p, S, T)$ is composed of the points lying in the Voronoi cell of p (p excluded), which are misclassified by S through the k NN rule. The implementation of the variant taking into account k NNs is the same as that shown in Fig. 5 with two differences: each entry $nearest[q]$ of the array $nearest$ stores the k objects of S closest to q, whereas S assigns q to the most frequent class label associated with the points in $nearest[q]$. Thus, Theorems 3.2 and 3.3 remain valid, but the space required is now $O(k|T|)$.

In Section 4.3, the behavior of this variant will be studied on some real-world domains.

4 Experimental Results

In this section, we present experimental results obtained by using the FCNN rule.

Next, the competence preservation algorithms compared are commented upon. As far as the FCNN rule is concerned, the algorithm described in Section 3 was used. As for the

TABLE 1
Experiments on Large Data Sets

	Method	Time	Subset size		Accuracy	Iterations	Complexity	Speed-up
Checkerboard	FCNN1	149	5,535	(0.55%)	99.77	149	1.44	11.2
	FCNN2	70	7,112	(0.71%)	99.75	25	1.45	44.4
	FCNN3	1,452	5,510	(0.55%)	99.77	5,510	1.37	10.4
	FCNN4	657	6,856	(0.69%)	99.76	6,856	1.37	85.6
	fMCNN	2,877	7,387	(0.74%)	99.74	3,975	1.60	–
	fCNN	2,748	7,866	(0.79%)	99.75	5	1.64	–
	TRAIN	–	–		99.80	–	–	–
Forest Cover Type	FCNN1	3,750	58,385	(10.05%)	85.64	59	1.61	17.0
	FCNN2	2,566	60,465	(10.41%)	84.99	27	1.59	21.8
	FCNN3	2,681	54,294	(9.35%)	84.78	54,294	1.59	21.1
	FCNN4	2,521	56,802	(9.78%)	84.60	56,802	1.59	40.8
	fMCNN	68,397	64,660	(11.13%)	85.39	29,234	1.84	–
	fCNN	35,019	64,741	(11.14%)	85.58	8	1.83	–
	TRAIN	–	–		87.51	–	–	–
DARPA 1998	FCNN1	33	3,365	(0.73%)	99.46	281	1.37	26.0
	FCNN2	23	4,064	(0.89%)	99.45	23	1.31	68.0
	FCNN3	308	3,205	(0.70%)	99.46	3,205	1.41	16.3
	FCNN4	232	3,946	(0.86%)	99.44	3,946	1.35	79.7
	fMCNN	858	4,730	(1.03%)	99.45	2,840	1.56	–
	fCNN	1,054	4,497	(0.98%)	99.38	13	1.65	–
	TRAIN	–	–		99.56	–	–	–

other methods, first of all, it must be noticed that the NNSRM rule is impracticable on large training sets since its first step consists of computing all the pairwise distances among training-set objects. Hence, we considered it only in experiments involving small-sized data sets. Furthermore, we note that the MCNN and CNN rules are slow if compared with FCNN. Thus, in order to improve the efficiency of these methods, we enhanced them by adopting the two following strategies: 1) *avoiding repeated distance computations* by maintaining the NN of each training-set point computed so far in an array of size $|T|$ and then comparing each point only with the incremental part of the training-set-consistent subset (whose definition varies depending on the method considered) during a generic iteration and 2) *exploiting the triangle inequality* technique described in Section 3.2 with the MCNN rule. Indeed, when set S_m becomes empty, the set C of the centroids computed by the MCNN method plays the same role as set ΔS in the FCNN rule. Since these implementations represent a major modification of the basic methods, in the following, the variant of the CNN rule augmented with strategy 1 is called fCNN, and the variant of the MCNN rule employing both strategies 1 and 2 is called fMCNN. It is worth pointing out that with respect to the subset computed and the number of iterations performed, the fCNN rule is equivalent to the CNN rule, and fMCNN is equivalent to MCNN.

The experiments are organized as follows:

Section 4.1 describes the experiments executed on three large training sets in order to study the scaling and learning behavior of the FCNN rule.

Next, Section 4.2 compares the competence preservation methods on small-sized real training sets. Both mutual condensation ratios and classification accuracies are measured and compared.

Finally, Section 4.3 experiments with the FCNN method on two pattern recognition domains, studies the behavior of the variant of the basic rule taking into account k neighbors, and compares the FCNN rule with hybrid instance-based learning algorithms.

The results of the experiments presented in this section are then discussed in Section 5.

In all of the experiments, we used the euclidean distance as the distance metric. All the experiments were executed on a Pentium IV 2.66-GHz-based machine with 1 Gbyte of main memory under the Windows operating system.

4.1 Large Data Set

We tested the training-set-consistent subset methods on the three following large data sets.

The *Checkerboard* data set is a synthetically generated 4×4 checkerboard data set partitioning points of the unit square into two classes composed of 1,000,000 points.

The *Forest Cover Type* data set[5] contains forest cover type data from the US Forest Service. It is composed of 581,012 tuples associated with 30 $\times$ 30 meter cells. Each tuple has 54 attributes, of which 10 are quantitative (for example, elevation, aspect, slope, and so forth), 4 are binary wilderness areas, and 40 are binary soil type variables. The data is partitioned into seven classes.

The US Defense Advanced Research Projects Agency's (DARPA) 1998 intrusion detection evaluation data set[6] consists of network intrusions simulated in a military network environment. The Transmission Control Protocol (TCP) connections have been elaborated to construct a data set of 23 numerical features, one of which identifies the kind of attack: *denial of service (DoS)*, *remote to local (R2L)*, *user to root (U2R)*, and *PROBING*. We used the TCP connections from five weeks of training data. The training set is composed of 458,301 objects partitioned into five classes (one representing normal data and the others associated with the different types of attack).

Table 1 summarizes the results of the experiments on the whole training set. The following are reported for each

5. http://kdd.ics.uci.edu/databases/covertype/covertype.html.
6. http://www.ll.mit.edu/IST/ideval/index.html.

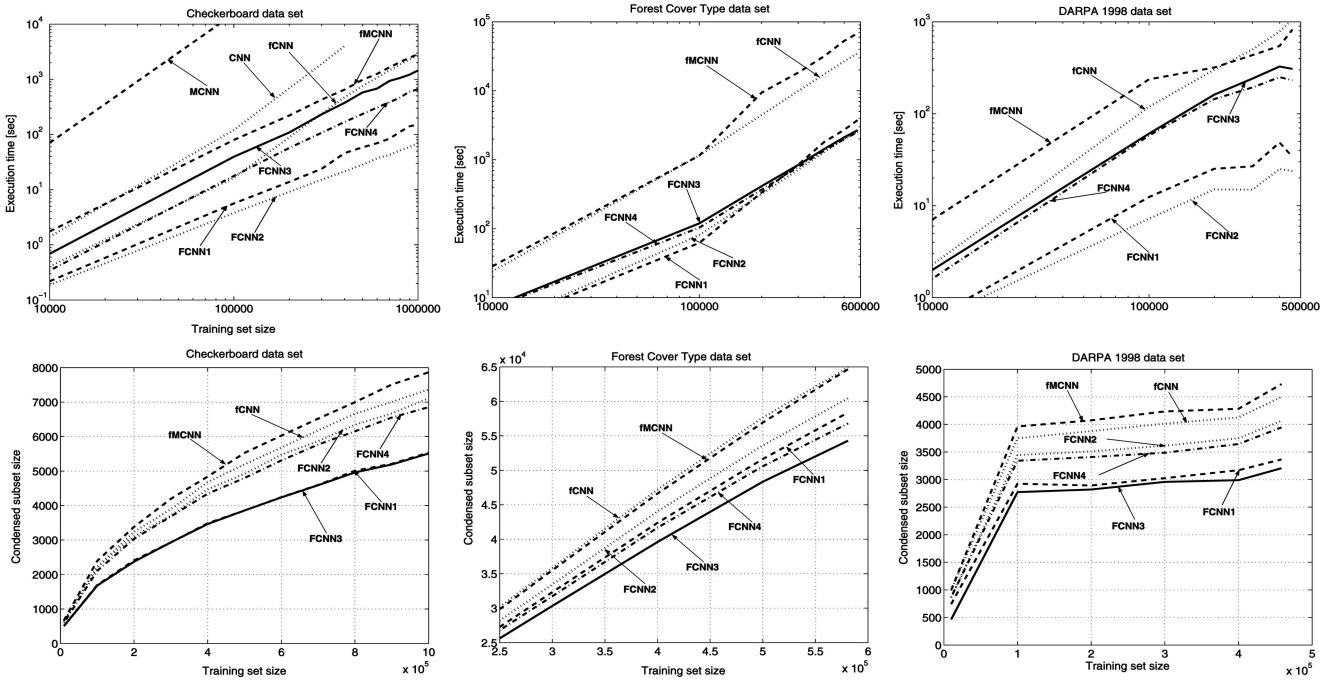

Fig. 6. **Large data sets:** scaling analysis.

experiment: the execution time in seconds, the number of objects composing the consistent subset (between parentheses there is the relative size of the subset), the test accuracy measured using the hold-out method, the number of iterations performed, the empirical complexity (defined next), and the speedup achieved by exploiting the triangle inequality (see Section 3.2).

The *empirical complexity* is the ratio $\log D / \log |T|$, where D denotes the number of distances computed by the method, and provides an estimate of its computational complexity. Although it provides a short summary, it is not sufficient to characterize the effective effort of the method, since the execution time is influenced also by the number of iterations and by the number of training-set passes per iteration.

The *speedup* (defined only for the FCNN methods) is defined as the ratio between the worst case number of distances computed by the method (see Theorem 3.3 and the related part in the text) and the number of distances actually computed by the method exploiting the triangle inequality. It was measured to verify the effectiveness of the triangle inequality technique described in Section 3.2.

Scaling analysis of the execution time and of the size of the consistent subset computed is reported in Fig. 6.

4.1.1 Execution Time

On the *Checkerboard* data set, the methods can be partitioned into three groups: FCNN1 and FCNN2 are very fast, FCNN3 and FCNN4 have higher time requirements than the previous two rules, and fMCNN and fCNN are slow. It is worth noting that the fMCNN and fCNN rules are 40 times slower than the FCNN2 rule. We also measured the execution time of the MCNN and CNN rules and, unfortunately, they turned out to be very slow. Indeed, MCNN required 131,402 sec to process 200,000 points, whereas CNN required 4,032 sec to process 400,000 points. Since these rules are too time demanding, we decided to disregard their analysis on larger samples. Moreover, they are no longer considered in subsequent experiments.

On the *Forest Cover Type* data set, the FCNN rules clearly outperformed both the fCNN and the fMCNN rules. In particular, on the whole training set, the FCNN2, FCNN3, and FCNN4 rules required about 2,500 sec, FCNN1 performed slightly worse, and the fCNN rule was more than 14 times slower than the FCNN rules, and fMCNN was about 27 times slower.

On the *DARPA 1998* data set, the fastest rules were FCNN2 and FCNN1, followed by FCNN4 and FCNN3 and, eventually, by fMCNN and fCNN. The first method was about 45 times faster than the latter.

4.1.2 Subset Size

On the *Checkerboard* data set, FCNN1 and FCNN3 computed the smallest subsets, whereas the FCNN2, FCNN4, CNN, and MCNN computed greater subsets, even though the MCNN computed a subset noticeably greater than those computed by any other method.

Fig. 7 shows the subset computed by the various methods on the whole data set (for clarity, only the square $[0.15, 0.35] \times [0.15, 0.35]$ is reported). The circles and crosses are the consistent subset points of the two classes. It can be noticed that the FCNN1 and FCNN3 rules select points very close to the boundaries between the checkerboard cells, whereas the other methods also contain points inside the cells, though the MCNN and CNN appears to have a high percentage of such points.

On *Forest Cover Type*, FCNN3 computed the smallest subset, followed by FCNN4, FCNN1, FCNN2, MCNN, and CNN. Notice that the subset computed by the FCNN3 rule contains 10,000 points less than the last two subsets.

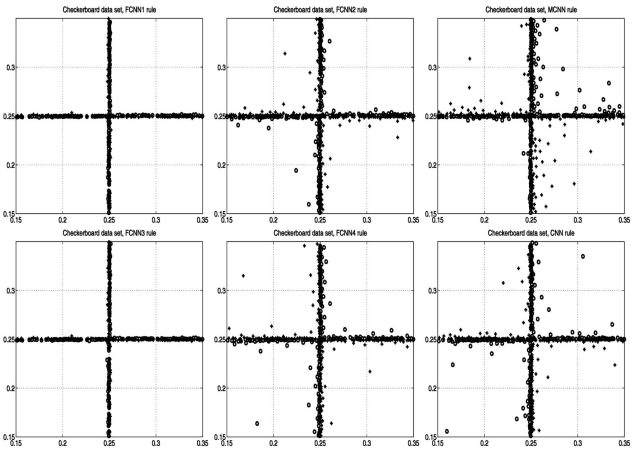

Fig. 7. **Checkerboard data set:** training-set-consistent subset computed.

As for *DARPA 1998*, three groups can be identified, that is, FCNN3 together with FCNN1, which reported the smallest subsets, FCNN4 together with FCNN2 and, finally, CNN together with MCNN. It is worth noting that the largest subset is about 1.5 times greater than those returned by the FCNN3 rule.

4.1.3 Iterations

As far as the number of iterations is concerned (see Table 1), it can be noticed that the behavior of the methods was the same in all the experiments. Indeed, the FCNN3 and FCNN4 rules in general execute a lot of iterations. In fact, since these rules add one point per iteration, the number of performed iterations is identical to the size of the training-set-consistent subset. Also, the MCNN rule adds at most m points per iteration, where m is the number of classes in T and, thus, the number of iterations it requires is greater than the number of subset points divided by the number of classes. The CNN rule always performs few iterations (less than 10) since it selects almost all the points of the subset during the first two iterations, and some additional iterations are then needed to assure consistency. However, this strategy has the drawback of producing a consistent subset noticeably larger than that returned by competitor methods. The FCNN2 rule always performs about a few tens of iterations, since it starts from the centroids of the classes and, due to the strategy adopted to select the representatives of the misclassified points, rapidly covers the regions of the feature space far from the centroids. Finally, the FCNN1 rule is sensitive to the complexity of the decision boundary. The more involved this boundary is, the more iterations are required by the rule. Indeed, an involved decision boundary can make it difficult to cover the feature space quickly, due to the strategy of selecting nearest Voronoi enemies as representatives of misclassified points. In summary, in general, the FCNN1 rule performs more iterations than the FCNN2 rule. Depending on the complexity of the decision boundary, it might execute from approximately the same number of iterations as the FCNN2 rule to some hundreds of iterations.

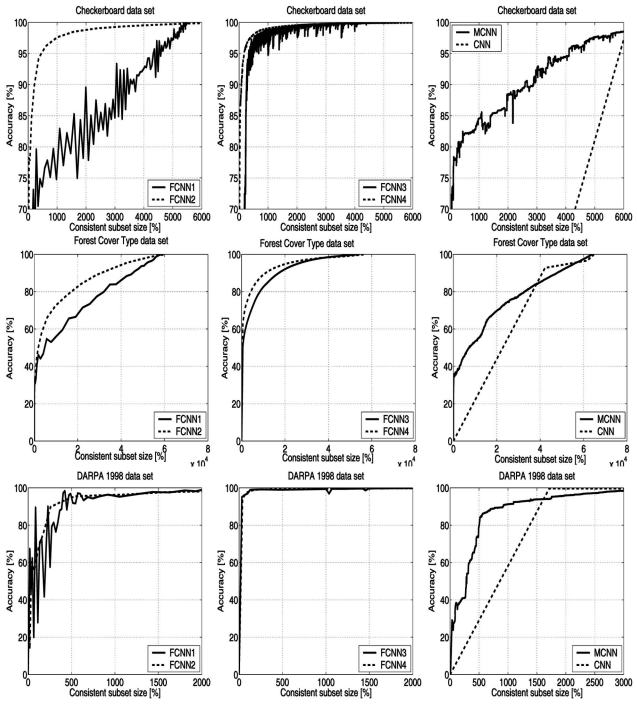

Fig. 8. **Large data sets:** training-set classification accuracy versus subset size.

4.1.4 Training-Set Accuracy

Fig. 8 shows the accuracy of the subset S_i, computed during the ith iteration of the algorithms, in classifying the overall training set T.

As for *Checkerboard*, for clarity, only values of accuracy between 70 percent and 100 percent and subsets containing less than 6,000 points are shown. On this training set, the curve of the FCNN1 rule oscillates sensibly and converges quite slowly. Differently, the curves of the FCNN2, FCNN3, and FCNN4 methods rapidly converge to high values of accuracy. The curve of the FCNN3 method also oscillates, but its fluctuation is more contained than that of the FCNN1 rule. It can be noticed that the area under the curve of the FCNN4 rule is the largest among the areas of all the methods (the first 178 points achieved an accuracy of 95.09 percent, 968 points, an accuracy of 99.00 percent, and 5,164 points, an accuracy of 99.90 percent), whereas the FCNN2 achieved the second largest area (366 points achieved an accuracy of 94.38 percent, 2,377 points, an accuracy 99.04 percent, and 6,727 points, an accuracy of 99.96 percent). The curve of the MCNN rule is similar to that of the FCNN1 rule, but it is less wavering. Finally, the curve of the CNN rule is the least accurate.

The above behavior is confirmed by the *Forest Cover Type* data set, where the curve of the FCNN4 method is the more accurate (the first 11,023 points achieved an accuracy of 90 percent, 20,738 points, an accuracy of 95 percent, and 48,009 points, an accuracy of 99 percent), and by the *DARPA 1998* data set, where the FCNN3 and FCNN4 showed the largest area (the curves of these two methods were smoothed for readability).

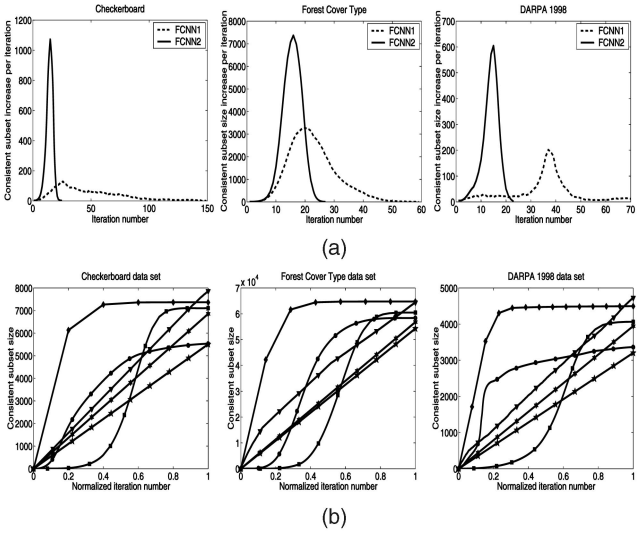

Fig. 9. **Large data sets:** points added per iteration versus iteration number (on the bottom: circle = FCNN1, square = FCNN2, pentagram = FCNN3, hexagram = FCNN4, triangle = MCNN, and diamond = CNN).

4.1.5 Course of the Subset Size

Fig. 9a shows the number $|\Delta S_i|$ of points added versus the iteration number i for the FCNN1 and FCNN2 methods. The curves show that during the initial and final iterations, the incremental sets include a relatively small number of points, whereas during the central iterations, the size of the incremental sets reaches a peak. Interestingly, the curve of the FCNN2 rule follows a Gaussian-like distribution, whereas the curve of the FCNN1 is also unimodal but less symmetrical. This behavior is observed in all the experiments.

Fig. 9b shows the size $|S_i|$ of the subset S_i versus the normalized iteration number $\frac{i}{i_{\max}}$. The curves of the FCNN3, FCNN4, and MCNN rules are straight lines, since they add almost a constant number of points per iteration (one point in the case of FCNN3 and FCNN4). The curve of CNN is a step, as almost all the points are selected during the first and the second iterations. Finally, the curves of the FCNN1 and FCNN2 rules resemble a Sigmoid as expected from the previous diagrams.

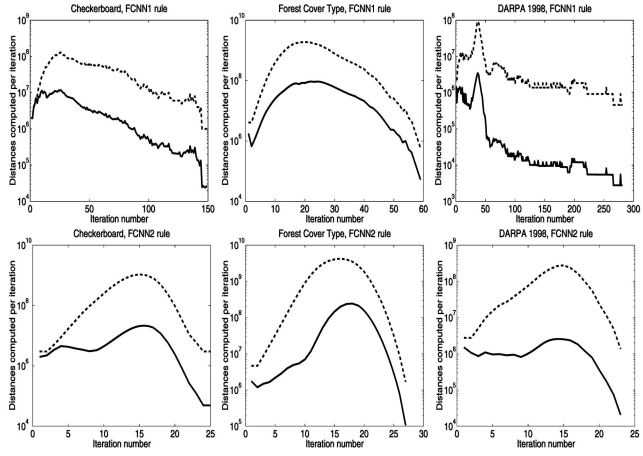

Fig. 10. **Large data sets:** distances computed versus iteration number (solid lines = distances computed by exploiting the triangle inequality, dashed lines = worst case number of distances to be calculated by the triangle inequality-based method).

TABLE 2
UCI ML Repository Data Sets Used in the Experiments

Data set name	*Abbreviation*	*Size*	*Features*	*Classes*
Bupa	BUP	345	6	2
Coil 2000	COI	4,992	85	2
Colon Tumor	COL	62	2,000	2
Echocardiogram	ECH	61	11	2
Ionosphere	ION	351	34	2
Iris	IRI	150	4	3
Image Segmentation	IMA	2,310	19	7
Pen Digits	PEN	7,494	16	10
Pima Indians Diabetes	PIM	768	8	2
Satellite Image	SAT	6,435	36	6
Spam Database	SPA	4,207	57	2
SPECT Heart Data	SPE	349	44	2
Vehicle	VEH	846	18	4
Wisconsin Breast Cancer	WBC	683	9	2
Wine	WIN	178	13	3
Wisc. Prognostic Breast Cancer	WPB	198	33	2

4.1.6 Distance Computation Savings

Fig. 10 shows the distances computed by the FCNN1 and FCNN2 methods versus the iteration number. The solid lines represent the number of distances actually calculated by the methods exploiting the triangle inequality, whereas the dashed lines represent the worst-case number of distances to be calculated by the same algorithms. It is clear from these figures that great savings are obtained, since the area between the solid and the dashed curves is at least one order of magnitude greater than the area under the solid curve. Similar curves were obtained also for the FCNN3 and FCNN4 rules. Table 1 summarizes the speedup obtained by the FCNN methods through the use of the triangle inequality.

4.1.7 Test Accuracy

We discussed in Section 1 techniques for improving the accuracy of NN classifiers. Here, we are mainly interested in comparing the performances of different competence preservation methods on the same data, rather than in improving the accuracy of the classifier. The test accuracy measured through the hold-out method is summarized in Table 1. The accuracy of the methods was always almost the same and, in two cases, identical to the accuracy of the whole training set, whereas on the *Forest Cover Type* data set, all the methods reported a sensible loss in accuracy with respect to the training set.

4.2 Small Data Sets

In these experiments, a number of small training sets are considered in order to compare the CNN, MCNN, and NNSRM rules with the FCNN rule.

The training sets employed are reported in Table 2. The data sets are from the University of California, Irvine (UCI) Machine Learning (ML) Repository.[7] Tenfold cross validation has been accomplished for each training set. Therefore, the measures reported in the following paragraphs concerning subset sizes, classification accuracies, and execution times are average values over the 10 executions. The

7. http://mlearn.ics.uci.edu/MLRepository.html.

TABLE 3
Small Data Sets: Training-Set-Consistent Subset Size and Percentage of Training-Set Instances Composing the Training-Set-Consistent Subset (between Parentheses)

	FCNN1		FCNN2		FCNN3		FCNN4		MCNN		CNN		NNSRM	
BUP	175	(56.45)	176	(56.77)	**168**	(54.19)	173	(55.81)	173	(55.81)	184	(59.26)	307	(99.03)
COI	904	(20.41)	907	(20.48)	**902**	(20.37)	913	(20.61)	909	(20.52)	1117	(24.86)	4372	(98.71)
COL	19	(34.55)	19	(34.55)	18	(32.73)	**17**	(30.91)	19	(34.55)	21	(37.63)	49	(89.09)
ECH	6	(11.11)	6	(11.11)	**5**	(9.26)	6	(11.11)	6	(11.11)	8	(14.57)	15	(27.78)
ION	60	(19.05)	62	(19.68)	59	(18.73)	58	(18.41)	**55**	(17.46)	73	(23.11)	309	(98.10)
IRI	15	(11.11)	14	(10.37)	16	(11.85)	14	(10.37)	**13**	(9.63)	16	(11.85)	—	
IMA	274	(13.18)	272	(13.08)	**245**	(11.78)	252	(12.12)	256	(12.31)	272	(13.08)	—	
PEN	300	(4.45)	284	(4.21)	286	(4.24)	**249**	(3.69)	253	(3.75)	311	(4.61)	—	
PIM	329	(47.61)	333	(48.19)	**316**	(45.73)	332	(48.05)	334	(48.34)	357	(51.65)	691	(100.00)
SAT	1051	(18.15)	1055	(18.22)	1007	(17.39)	**983**	(16.97)	1012	(17.48)	1128	(19.48)	—	
SPA	849	(22.42)	869	(22.95)	819	(21.63)	805	(21.26)	**800**	(21.13)	961	(25.38)	3774	(99.68)
SPE	98	(31.21)	104	(33.12)	**93**	(29.62)	96	(30.57)	94	(29.94)	107	(34.07)	257	(81.85)
VEH	401	(52.69)	398	(52.30)	385	(50.59)	**382**	(50.20)	390	(51.25)	405	(53.19)	—	
WBC	55	(8.96)	55	(8.96)	**54**	(8.79)	**54**	(8.79)	**54**	(8.79)	64	(10.41)	511	(83.22)
WIN	63	(39.38)	64	(40.00)	**62**	(38.75)	63	(39.38)	64	(40.00)	65	(40.57)	—	
WPB	84	(47.19)	85	(47.75)	**79**	(44.38)	83	(46.63)	84	(47.19)	90	(50.51)	175	(98.31)
FCNN1	—		1.00		1.04		1.04		1.04		0.90		0.32	
FCNN2	1.00		—		1.05		1.05		1.04		0.91		0.33	
FCNN3	0.96		0.95		—		1.00		0.99		0.87		0.31	
FCNN4	0.96		0.96		1.00		—		0.99		0.87		0.32	
MCNN	0.97		0.96		1.01		1.01		—		0.87		0.32	
CNN	1.11		1.10		1.16		1.15		1.15		—		0.37	
NNSRM	3.09		3.04		3.24		3.16		3.13		2.69		—	

TABLE 4
Small Data Sets: Classifier Accuracy

	FCNN1	FCNN2	FCNN3	FCNN4	MCNN	CNN	NNSRM	TRAIN
BUP	58.53 ± 0.88	59.71 ± 0.80	59.41 ± 0.79	58.23 ± 0.81	56.18 ± 0.84	56.47 ± 0.71	58.23 ± 0.57	57.94 ± 0.59
COI	85.73 ± 0.20	85.45 ± 0.20	85.55 ± 0.16	85.85 ± 0.15	85.95 ± 0.40	86.61 ± 0.16	89.47 ± 0.16	89.39 ± 0.16
COL	75.00 ± 1.71	78.33 ± 1.50	76.67 ± 2.00	78.33 ± 1.68	76.67 ± 1.34	80.00 ± 1.79	78.33 ± 0.76	80.00 ± 1.00
ECH	95.00 ± 0.76	93.33 ± 0.82	95.00 ± 0.76	93.33 ± 0.82	91.67 ± 1.05	90.00 ± 1.11	96.67 ± 0.67	95.00 ± 0.76
ION	85.43 ± 0.69	86.00 ± 0.61	86.00 ± 0.61	86.29 ± 0.55	86.29 ± 0.75	86.00 ± 0.56	86.86 ± 0.69	86.86 ± 0.69
IRI	93.33 ± 0.52	92.00 ± 0.50	93.33 ± 0.52	92.67 ± 0.55	92.67 ± 0.72	95.33 ± 0.52	—	95.33 ± 0.43
IMA	94.81 ± 0.18	95.71 ± 0.22	95.02 ± 0.22	94.63 ± 0.24	95.33 ± 0.44	94.63 ± 0.19	—	96.36 ± 0.16
PEN	98.69 ± 0.03	98.75 ± 0.03	98.60 ± 0.03	98.44 ± 0.04	98.64 ± 0.18	98.60 ± 0.03	—	99.44 ± 0.03
PIM	67.11 ± 0.29	67.63 ± 0.42	66.45 ± 0.22	66.71 ± 0.34	65.92 ± 0.61	65.39 ± 0.37	69.21 ± 0.47	69.21 ± 0.47
SAT	88.48 ± 0.15	88.44 ± 0.14	88.20 ± 0.12	88.68 ± 0.14	88.90 ± 0.40	88.69 ± 0.16	—	90.64 ± 0.13
SPA	86.05 ± 0.22	86.17 ± 0.20	85.88 ± 0.29	86.17 ± 0.23	85.57 ± 0.45	86.45 ± 0.20	89.79 ± 0.14	89.79 ± 0.14
SPE	83.53 ± 0.61	85.00 ± 0.55	84.41 ± 0.67	82.94 ± 0.72	82.35 ± 0.82	83.82 ± 0.67	86.47 ± 0.46	86.47 ± 0.46
VEH	62.38 ± 0.55	64.76 ± 0.57	63.33 ± 0.79	62.14 ± 0.61	62.02 ± 0.85	62.38 ± 0.72	—	65.36 ± 0.68
WBC	93.53 ± 0.21	93.68 ± 0.25	93.53 ± 0.26	93.68 ± 0.28	93.82 ± 0.36	93.82 ± 0.13	95.88 ± 0.18	96.03 ± 0.13
WIN	76.47 ± 0.79	75.88 ± 0.76	74.71 ± 0.91	77.06 ± 0.72	74.71 ± 0.94	74.71 ± 0.87	—	77.06 ± 0.85
WPB	68.95 ± 0.72	68.42 ± 1.15	68.42 ± 0.88	68.42 ± 1.13	67.37 ± 1.00	68.42 ± 1.00	70.53 ± 0.75	71.05 ± 0.79
mean 1	82.06	82.45	82.16	82.10	81.50	81.96	—	**84.12**
mean 2	79.89	80.37	80.13	80.00	79.18	79.70	82.14	**82.17**

NNSRM rule was tested only on training sets having two class labels, since the implementation we had available supports only this kind of data sets.

We start commenting on the results shown in Table 3. This table reports the size of the training-set-consistent subsets (first line) together with the percentage of objects included in each subset (second line). At the bottom of the table, the achieved compression ratios are compared.

Since the size of the smallest training-set-consistent subset is an intrinsic property of the training set, to compare the various methods, we measured the ratios $\frac{size_{m_1}\%}{size_{m_2}\%}$, where m_1 and m_2 denote two methods, and $size_{m_i}\%$ denotes the percentage of training-set objects composing the subset computed by the method m_i. In particular, the entry in row m_r and column m_c at the bottom of the table represents the geometric mean of the ratios $\frac{size_{m_r}\%}{size_{m_c}\%}$ achieved in the experiments reported at the top of the table.

As for the size of the subset, on these training sets, the FCNN3 and FCNN4 rules performed better than any other method. As for the FCNN1 and the FCNN2 methods, they computed a subset that is, on the average, respectively, four and five percent larger than that computed by the FCNN4 method. This difference can be explained by noticing that the FCNN3 and FCNN4 rules are more accurate than the FCNN1 and FCNN2 rules in the choice of the prototypes to add to the current subset. The MCNN method performed even well: indeed, the increase in size with respect to the FCNN3 method is, on the average, one percent. The increase in size with respect to the FCNN3 and FCNN4 of the CNN rule is noticeable, since it amounts to 16 percent. Finally, it can be observed that the NNSRM rule contains a huge percentage of the training-set objects. Thus, its increase in size with respect to the other methods is very high, namely, more than 200 percent in almost all the experiments.

Table 4 shows the classification accuracy measured using tenfold cross validation. The last column is the classification achieved when all the training-set objects are used as a reference set during classification. At the bottom, the mean over all the experiments (mean 1) and the mean over the experiments concerning the NNSRM rule (mean 2) are

TABLE 5
Small Data Sets: Relative Execution Time

	FCNN1	FCNN2	FCNN3	FCNN4	MCNN	CNN	NNSRM
FCNN1	—	1.07	0.63	0.79	0.03	0.39	0.30
FCNN2	0.93	—	0.56	0.70	0.02	0.31	0.24
FCNN3	1.59	1.80	—	1.25	0.05	0.61	0.42
FCNN4	1.27	1.44	0.80	—	0.04	0.49	0.36
MCNN	33.17	44.78	20.90	26.19	—	12.83	8.16
CNN	2.59	3.24	1.63	2.04	0.08	—	0.62
NNSRM	3.28	4.22	2.37	2.76	0.12	1.62	—

reported. We can observe that all of the methods present a loss of classification accuracy with respect to the use of the overall training set. This loss ranges from -1.5 percent of the FCNN2 rule to -2.0 percent of the CNN rule. Furthermore, we note that the NNSRM rule exhibits a negligible loss in accuracy. This can be explained by noticing that the subset computed by this rule contains a high percentage of training-set objects, almost all the object in most cases; thus, the small loss in accuracy is not repaid by a significant reduction of the reference set.

Finally, Table 5 summarizes the execution times. In almost all cases, the times amount to a small fraction of 1 sec or to a few seconds. Thus, we reported only the relative speeds: The entry in row m_r and column m_c of the table represents the geometric mean of the ratios $\frac{time_{m_r}}{time_{m_c}}$, where $time_{m_i}$ denotes the execution time of the method m_i. It can be observed that on these data sets, FCNN2 is the fastest method, followed by the FCNN1 method, whereas CNN, NNSRM, and MCNN are the slowest methods. In particular, CNN is two and half times slower than FCNN2, NNSRM is three times slower, and MCNN is 30 times slower. It should be noticed that the last result is due to the presence of some outlying data sets on which the MCNN rule performs badly (that is, `COI`, `SAT`, and `SPA`).

4.3 Comparison with Hybrid Methods

In this section, some experiments on the *MNIST* and *MIT Face* databases are described. These two real-world databases are considered good testbeds for learning techniques and pattern recognition methods. Thus, the goal of this section is to test the applicability of the FCNN rule on some typical pattern recognition domains and to compare the FCNN methods with *hybrid* instance-based learning methods on some real-world applications.

The *MNIST* database[8] of handwritten digits has a training set of 60,000 examples and a test set of 10,000 examples. The digits have been size-normalized and centered in a 28×28 image. Each example is composed of 784 features, each associated with a distinct pixel of the image (pixel values range from 0 to 255). Examples are partitioned in 10, about equally sized, classes. The *MIT Face* database[9] is a database of faces and nonfaces, which has been used extensively at the Center for Biological and Computational Learning, MIT. This database has been partitioned in a training set of 25,317 face and 2,603 nonface examples and in a test set of 2,804 face and 298 nonface examples. Each example (a 19×19 image) consists of 361 features whose values range between 0 and 1.

8. http://yann.lecun.com/exdb/mnist/.
9. http://www.ai.mit.edu/projects/cbcl.

TABLE 6
Execution Time (Seconds), Subset Size, and Test Accuracy of the FCNN Rules on the (a) *MNIST* and (b) *MIT Face* Data Sets

	Execution time	Subset size		Test accuracy
FCNN1	1,031.5	6,275	(10.5%)	94.5
FCNN2	959.3	6,130	(10.2%)	94.5
FCNN3	1,244.6	5,704	(9.5%)	94.2
FCNN4	1,146.7	5,503	(9.2%)	94.2

(a)

	Execution time	Subset size		Test accuracy
FCNN1	26.5	1,627	(5.8%)	96.8
FCNN2	25.2	1,557	(5.5%)	96.8
FCNN3	95.0	1,593	(5.7%)	96.5
FCNN4	91.2	1,596	(5.7%)	96.5

(b)

In all of the experiments, the euclidean distance was used.

Experimental results concerning the FCNN method are summarized in Table 6. As for the *MNIST* data set (see Table 6a), all the FCNN methods employed from 15 to 20 minutes to compute a consistent subset including about 10 percent of the data set objects. The test accuracy of the FCNN rule is 94.5 percent (in [23], a k-NN classifier using the whole training set, the parameter k set to three, and the euclidean distance achieved 5.0 percent test error rate). On the *MIT Face* data set, FCNN1 and FCNN2 employed about 25 sec, whereas FCNN3 and FCNN4, about 95 sec. The extracted subset is, in all cases, composed of 5.5 percent of data set objects. The test accuracy is good, being close to 97 percent.

Fig. 11 shows the results obtained by using the variant of the FCNN rule taking into account k NNs (see Section 3.3) for k ranging from one to seven. By using $k = 3$, the execution time (Fig. 11a) increased with respect to the case of $k = 1$. For greater values of k, the growth slowed down, and the time remained substantially unchanged. As for the subset size (Fig. 11b), it remained almost the same for *MNIST* and slightly decreased for *MIT Face*. In the latter case, the reduction achieved by the FCNN3 and FCNN4 methods is substantial. Finally, by increasing the number k of considered NNs, the test accuracy (Fig. 11c) improved. For example, the accuracy of the FCNN1 rule increased by 1.5–2.0 percent for $k = 7$.

The FCNN rule was compared with the DROP3 [32], ELGrow, and Explore [9] hybrid instance-based learning algorithms.[10] DROP3 was observed to have the best mix of storage reduction and generalization among the *Decremental Reduction Optimization Procedures,* DROP1–DROP5, presented in [32]. The ELGrow and Explore methods are able to achieve high storage reduction but are not as accurate as the DROP methods.

These methods were compared with the FCNN rules on the data sets *MNIST* and *MIT Face* (see Fig. 12). The whole training set and some random samples having an increasing size were considered to study the scaling behavior. If required by the method, the parameter k was set to three as done in [32].

10. The implementations of these algorithms as available at the location ftp://axon.cs.byu.edu/pub/randy/ml/drop were used.

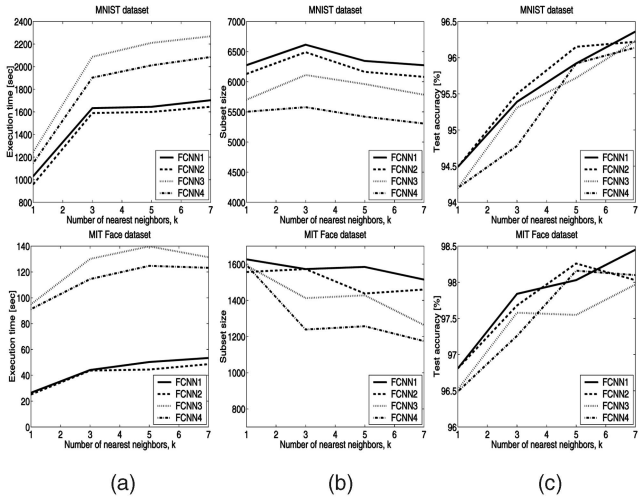

Fig. 11. The effect of considering k NNs on the *MNIST* and *MIT Face* data sets.

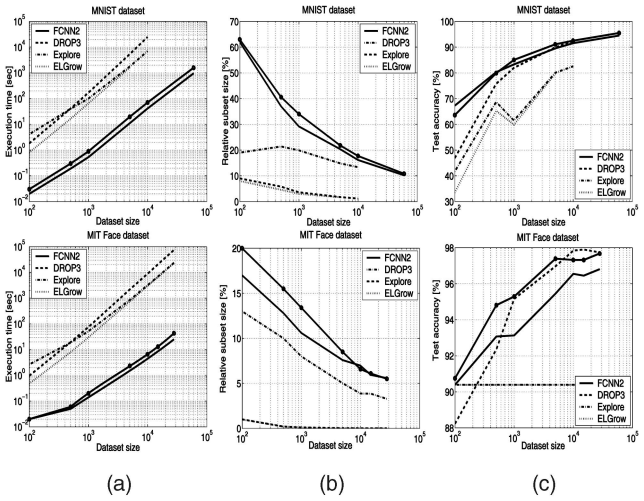

Fig. 12. Comparison with hybrid methods.

Fig. 12a shows the execution time versus the training-set size of the FCNN2, DROP3, ELGrow, and Explore algorithms on the two above-described data sets. As already stated, the FCNN rule employed about 1,000 sec on the whole *MNIST* data set. On this data set, the other methods were very slow, and it was decided to stop their execution. Thus, the largest sample considered for all methods included 10,000 examples. In this case, DROP3 required about 25,000 sec, ELGrow and Explore, about 7,000 sec, and FCNN, about 45 sec. On the whole *MIT Face* training set, DROP3 employed about 75,000 sec, ELGrow and Explore, about 23,000 seconds, and FCNN, about 25 sec. It is clear that as far as the learning speed is concerned, there is a difference of at least three orders of magnitude between the FCNN and the other competence enhancement methods on these medium-sized data sets. On larger data sets the hybrid methods are impractical.

Fig. 12b shows the relative subset size versus the training-set sample size. The competence enhancement methods confirmed their expected behavior, since ELGrow and Explore achieved very high data reduction. FCNN has the smallest reduction ratio, but except for very small sample sets, the size of its consistent subset is very close to that of DROP3, although slightly larger.

Finally, Fig. 12c shows the test accuracy achieved by using the subset computed by the various condensation methods. The ELGrow and Explore methods are the less accurate methods. As observed in [32], their aggressive storage reduction strategy may worsen their generalization capability. For example, on the *MIT Face* data set, the subsets computed by ELGrow and Explore always misclassify the examples of the minority class. The loss in accuracy of ELGrow/Explore with respect to FCNN on the *MNIST* is about 10 percent. On the contrary, the DROP3 method confirms its good generalization capabilities due to a noise-filtering pass. It must be pointed out that FCNN performed remarkably well compared to instance-based competence enhancement methods. Indeed, other than performing better than ELGrow/Explore, it has the same accuracy as DROP3 on the *MNIST* data set, whereas the difference in accuracy between these two methods is about 1 percent on the *MIT Face* data set.

Since DROP3 incorporates a noise-filtering pass based on removing instances misclassified by its k NNs ($k = 3$ in the experiments), Fig. 12 reports also the behavior of the FCNN2 rule taking into account $k = 3$ neighbors (the solid dotted line). By considering three NNs, as already observed, FCNN slightly increases its execution time, with no appreciable difference on the size of the subset, and the accuracy improves and is identical to that of noise-filtering methods.

5 Discussion and Conclusions

This work introduces a novel algorithm, called the FCNN rule, for computing a training-set-consistent subset for the NN rule.

The algorithm starts by selecting the centroid of each training-set class and, then, until consistency is achieved, selects for insertion a representative of the misclassified points of each Voronoi cell induced by the current subset. Four variants of the basic method are presented, called FCNN1–4 rules. In particular, the FCNN1 and FCNN2 rules augment the subset with all such representatives, whereas the FCNN3 and FCNN4 rules select only a representative per iteration. The FCNN1 (respectively, FCNN2) and the FCNN3 (respectively, FCNN4) rules are based on the same definition of a representative.

Each of these rules has strengths and weaknesses that, generally speaking, can be summarized as follows:

FCNN3 and FCNN4 are more careful in the choice of the points to select for insertion and, hence, FCNN3 (respectively, FCNN4) returns a subset smaller than FCNN1 (respectively, FCNN2).

On the contrary, FCNN1 and FCNN2 execute few iterations and are noticeably faster, with FCNN2 being the fastest. This can be explained by noticing that it appears to be a little sensitive to the complexity of the decision boundary. Indeed, it rapidly covers regions of the space far from the centroids of the classes and always performs about a few tens of iterations.

FCNN1 is slightly slower than FCNN2, and it requires more iterations, up to a few hundreds. However, together with the FCNN3, it is likely to select points very close to the decision boundary and hence may return a subset smaller than that of FCNN2. From what has been stated above, as far as the subset computed is concerned, FCNN1 and FCNN3 are probably preferable when the classes are well

separated (for an example, see the subsets of the *Checkerboard* data set in Fig. 7).

The FCNN4 rule presents the greatest area under the curve of the training set accuracy versus the current subset size. Hence, it can be stopped early, when a satisfactory degree of training-set accuracy is achieved, to obtain a noticeably smaller condensed subset (as an example, the first thousand points selected on the *Checkerboard* data set guarantee a 99 percent accuracy on the training set, though the final set includes about 7,000 points). This strategy can be profitably used also with the other FCNN rules.

The FCNN rule is order independent, its worst-case time complexity is quadratic with an often small constant prefactor, and it permits the triangle inequality to be effectively exploited to reduce the computational effort (as witnessed in Fig. 10). It was tested on large and high-dimensional training sets with very good results. For example, on an ordinary personal computer, it was able to compute a consistent subset (including about 4,000 out of about half a million points) of the *DARPA 1998* data set in more or less 20 sec.

The FCNN rule was compared with the CNN, MCNN, and NNSRM algorithms.

Comparison on small data yielded the following outcomes. With regard to classification accuracy, there are no appreciable differences, since all the methods present a comparable loss (about 1.5 percent) with respect to the whole training set. The only exception is NNSRM, which scored the same accuracy as the training set. Nevertheless, as far as the condensation size is concerned, MCNN is the only rule comparable with FCNN. Indeed, CNN computes a subset from 10 percent to 15 percent larger than that of the FCNN, whereas NNSRM tends to select almost all the training-set objects and, hence, the slightly better accuracy it offers is not repaid by an appreciable reduction of the training set.

Existing condensation algorithms are too slow to be applicable to large data sets, since their complexity is superquadratic or even cubic. Thus, improved implementations of the CNN and MCNN rules, called, respectively, fCNN and fMCNN, are introduced here and compared to FCNN. Experiments show that the FCNN rule outperforms even these enhanced methods while guaranteeing the same accuracy as the training set. It is worth noticing that as far as the classification accuracy is concerned, this is the expected behavior of the compared methods, since their goal is to preserve the competence of the whole training set.

The observed superior learning speed is also substantiated by the learning behavior comparison. Indeed, the FCNN rules approach consistency more rapidly (see Fig. 8) and return a smaller subset.

A variant of FCNN taking into account k NNs has been introduced and experimented on two real data sets. By increasing k, the execution time slightly increases, the size of subset remains almost the same, and the test accuracy may improve.

The comparison with hybrid methods on medium-sized (including from 10,000 to 20,000 examples) real-world (character and face recognition) high-dimensional (up to 784 features) data sets showed that FCNN is at least three orders of magnitude faster and that it achieves both a reduction ratio and a test accuracy comparable to DROP3 (see Fig. 12), although DROP3 incorporates a data-set-editing step able to filter out noise and producing a slightly smaller subset. ELGrow and Explore perform noticeably worse than FCNN in terms of accuracy but have higher reduction ratios. From the scaling analysis, it is clear that hybrid methods are impractical on larger data sets. Thus, in real-world domains, FCNN can compute a model comparable to that of hybrid algorithms. FCNN is efficient even if the collection of data to be processed is very large, whereas other algorithms may be not able to manage the same data set sizes in a reasonable amount of time.

To conclude, it is worth pointing out that this is the first work providing condensation algorithms for the NN rule that can be efficiently applied to large data sets.

APPENDIX

PROOF OF THEOREM 3.2

1) First, we note that the time required to compute the class centroids of the training set and to perform a single iteration of the algorithm is upper bounded by a polynomial in the number $|T|$ of training-set instances.

During a generic iteration of the algorithm, at least one element of $T - S$ is selected and inserted in S; otherwise, the algorithm stops. Let us call m the number of class labels in the training set. Since subset S contains initially m elements, in the worst case, the algorithm performs $|T| - m + 1$ iterations.

With the training set T being composed of a finite number of elements, it can be concluded that the overall time required by the method is finite.

2) The property is guaranteed by the termination condition. Indeed, the algorithm stops when the set ΔS becomes empty, that is, if the condition of Theorem 3.1 is satisfied.

3) First of all, we show that given a (not necessarily proper) subset X of T, it holds that

- the set $Centroids(X)$, composed of the class centroids of X, is order independent,
- point $nn(p, X)$ of X closest to p is order independent,
- for each $p \in X$, the Voronoi cell $Vor(p, X, T)$ is order independent, and
- point p^* of X such that $|Voren(p^*, X, T)|$ is maximum is order independent.

a) Consider the set $Centroids(X)$. Let $l_1, \ldots, l_m$ be the class labels in X and let X_i be the subset of X composed of the points of the class l_i. Then, the set $Centroids(X)$ has the form $\{c_1, \ldots, c_m\}$, where each c_i is the centroid of X_i, that is, the point of X_i that is closest to the geometrical center C_i of X_i. Clearly, C_i is unique. As for c_i, usually, it is unique, but ties are possible and are solved in favor of the lexicographically smallest point.

Let $q = (q_1, \ldots, q_d)$ and $r = (r_1, \ldots, r_d)$ be two points. Then, q is lexicographically smaller than r if there is an integer j, $1 \leq j \leq d$, such that $q_1 = r_1, \ldots, q_{j-1} = r_{j-1}$ and $q_j < r_j$. If there is more than one lexicographically smallest point, then it is the case that these points are identical, no matter which is selected. Thus, the set computed is independent of the order in which X_i is processed.

b) Consider point $nn(p, X)$. Usually, the point of X closest to p is unique, but ties are possible and are solved in favor of the lexicographically smallest point. Nevertheless, it can be that there are two lexicographically smallest points q and r having different class labels l_q and l_r. In this case, q is selected, provided that $l_q < l_r$. Hence, the point selected is independent of the order in which X is processed.

c) Usually, for each point $q \in T$, the NN of q in X is unique and, thus, also the Voronoi cell that it belongs to, but ties are possible and are solved as explained in point b) above.

d) We have already seen that the set $Vor(p, X, T)$ is order independent. Thus, for each $p \in X$, sets $Voren(p, X, T)$ are order independent. Usually, point $p^* \in X$ maximizing $|Voren(p^*, X, T)|$ is unique, but ties are possible and are solved in favor of the lexicographically smallest point.

It remains to be shown that for each iteration $i \geq 0$, the set of points ΔS_i selected by the algorithm is independent of the order in which training-set elements are processed. The proof now proceeds by induction on the iteration number.

Let $i = 0$, then $S_0 = \emptyset$, and ΔS_0 is the set $Centroids(T)$. Hence, the result follows from point a).

Let $i > 0$ and assume that $\Delta S_0, \ldots, \Delta S_{i-1}$ are order independent. Then, $S_i = S_{i-1} \cup \Delta S_{i-1} = S_{i-2} \cup \Delta S_{i-2} \cup \Delta S_{i-1} = \ldots = \Delta S_0 \cup \ldots \cup \Delta S_{i-1}$ is order independent. Furthermore, for each $p \in S_i$, i) $Voren(p, S_i, T)$ is order independent, being a particular subset of $Vor(p, S_i, T)$, and ii) $rep(p, Voren(p, S_i, T))$ is order independent, as both $nn(p, Voren(p, S_i, T))$ and $nn(p, Centroids(Voren(p, S_i, T)))$ are order independent. Hence, it can be concluded that ΔS_i is order independent, and the result follows. □

ACKNOWLEDGMENTS

The author would like to thank the anonymous reviewers for their useful comments.

REFERENCES

[1] D.W. Aha, "Editorial," *Artificial Intelligence Rev.,* special issue on lazy learning, vol. 11, nos. 1-5, pp. 7-10, 1997.
[2] D.W. Aha, D. Kibler, and M.K. Albert, "Instance-Based Learning Algorithms," *Machine Learning,* vol. 6, pp. 37-66, 1991.
[3] E. Alpaydin, "Voting over Multiple Condensed Nearest Neighbors," *Artificial Intelligence Rev.,* vol. 11, pp. 115-132, 1997.
[4] F. Angiulli, "Fast Condensed Nearest Neighbor Rule," *Proc. 22nd Int'l Conf. Machine Learning (ICML '05),* pp. 25-32, 2005.
[5] S. Bay, "Combining Nearest Neighbor Classifiers through Multiple Feature Subsets," *Proc. 15th Int'l Conf. Machine Learning (ICML '98),* 1998.
[6] S. Bay, "Nearest Neighbor Classification from Multiple Feature Sets," *Intelligent Data Analysis,* vol. 3, pp. 191-209, 1999.
[7] B. Bhattacharya and D. Kaller, "Reference Set Thinning for the *k*-Nearest Neighbor Decision Rule," *Proc. 14th Int'l Conf. Pattern Recognition (ICPR '98),* 1998.
[8] H. Brighton and C. Mellish, "Advances in Instance Selection for Instance-Based Learning Algorithms," *Data Mining and Knowledge Discovery,* vol. 6, no. 2, pp. 153-172, 2002.
[9] R.M. Cameron-Jones, "Instance Selection by Encoding Length Heuristic with Random Mutation Hill Climbing," *Proc. Eighth Australian Joint Conf. Artificial Intelligence,* pp. 99-106, 1995.
[10] T.M. Cover and P.E. Hart, "Nearest Neighbor Pattern Classification," *IEEE Trans. Information Theory,* vol. 13, no. 1, pp. 21-27, 1967.
[11] B. Dasarathy, *Nearest Neighbor (NN) Norms-NN Pattern Classification Techniques.* IEEE CS Press, 1991.
[12] B. Dasarathy, "Minimal Consistent Subset (MCS) Identification for Optimal Nearest Neighbor Decision Systems Design," *IEEE Trans. Systems, Man, and Cybernetics,* vol. 24, no. 3, pp. 511-517, 1994.
[13] B. Dasarathy, "Nearest Unlike Neighbor (NUN): An Aid to Decision Confidence Estimation," *Optical Eng.,* vol. 34, pp. 2785-2792, 1995.
[14] F.S. Devi and M.N. Murty, "An Incremental Prototype Set Building Technique," *Pattern Recognition,* vol. 35, no. 2, pp. 505-513, 2002.
[15] P. Devijver and J. Kittler, "On the Edited Nearest Neighbor Rule," *Proc. Fifth Int'l Conf. Pattern Recognition (ICPR '80),* pp. 72-80, 1980.
[16] L. Devroye, "On the Inequality of Cover and Hart in Nearest Neighbor Discrimination," *IEEE Trans. Pattern Analysis and Machine Intelligence,* vol. 3, pp. 75-78, 1981.
[17] L. Devroye, L. Gyorfy, and G. Lugosi, *A Probabilistic Theory of Pattern Recognition.* Springer, 1996.
[18] K. Fukunaga and L.D. Hostetler, "*k*-Nearest-Neighbor Bayes-Risk Estimation," *IEEE Trans. Information Theory,* vol. 21, pp. 285-293, 1975.
[19] V. Gaede and O. Günther, "Multidimensional Access Methods," *ACM Computing Surveys,* vol. 30, no. 2, pp. 170-231, 1998.
[20] W. Gates, "The Reduced Nearest Neighbor Rule," *IEEE Trans. Information Theory,* vol. 18, no. 3, pp. 431-433, 1972.
[21] P.E. Hart, "The Condensed Nearest Neighbor Rule," *IEEE Trans. Information Theory,* vol. 14, no. 3, pp. 515-516, 1968.
[22] B. Karaçali and H. Krim, "Fast Minimization of Structural Risk by Nearest Neighbor Rule," *IEEE Trans. Neural Networks,* vol. 14, no. 1, pp. 127-134, 2003.
[23] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proc. IEEE,* vol. 86, no. 11, pp. 2278-2324, 1998.
[24] C.L. Liu and M. Nakagawa, "Evaluation of Prototype Learning Algorithms for Nearest-Neighbor Classifier in Application to Handwritten Character Recognition," *Pattern Recognition,* vol. 34, no. 3, pp. 601-615, 2001.
[25] G.L. Ritter, H.B. Woodruff, S.R. Lowry, and T.L. Isenhour, "An Algorithm for a Selective Nearest Neighbor Decision Rule," *IEEE Trans. Information Theory,* vol. 21, pp. 665-669, 1975.
[26] C. Stanfill and D. Waltz, "Towards Memory-Based Reasoning," *Comm. ACM,* vol. 29, pp. 1213-1228, 1994.
[27] C. Stone, "Consistent Nonparametric Regression," *Annals of Statistics,* vol. 8, pp. 1348-1360, 1977.
[28] G. Toussaint, "Proximity Graphs for Nearest Neighbor Decision Rules: Recent Progress," *Proc. 34th Symp. Interface of Computing Science and Statistics (Interface '02),* Apr. 2002.
[29] I. Watson and F. Marir, "Case-Based Reasoning: A Review," *The Knowledge Eng. Rev.,* vol. 9, no. 4, 1994.
[30] G. Wilfong, "Nearest Neighbor Problems," *Int'l J. Computational Geometry & Applications,* vol. 2, no. 4, pp. 383-416, 1992.
[31] D.L. Wilson, "Asymptotic Properties of Nearest Neighbor Rules Using Edited Data," *IEEE Trans. Systems, Man, and Cybernetics,* vol. 2, pp. 408-420, 1972.
[32] D.R. Wilson and T.R. Martinez, "Reduction Techniques for Instance-Based Learning Algorithms," *Machine Learning,* vol. 38, no. 3, pp. 257-286, 2000.

Fabrizio Angiulli received the Laurea degree in computer engineering from the University of Calabria, Italy, in 1999. From January 2001 to August 2006, he was with the Institute of High Performance Computing and Networking, Italian National Research Council (ICAR-CNR). Since September 2006, he has been an assistant professor in the Department of Electronics, Informatics, and Systems (DEIS), University of Calabria. His research interests include artificial intelligence, data mining, machine learning, and databases.

Fast Condensed Nearest Neighbor Rule

Fabrizio Angiulli ANGIULLI@ICAR.CNR.IT
ICAR-CNR, Via Pietro Bucci 41C, 87036 Rende (CS), Italy

Abstract

We present a novel algorithm for computing a training set consistent subset for the nearest neighbor decision rule. The algorithm, called FCNN rule, has some desirable properties. Indeed, it is order independent and has subquadratic worst case time complexity, while it requires few iterations to converge, and it is likely to select points very close to the decision boundary. We compare the FCNN rule with state of the art competence preservation algorithms on large multidimensional training sets, showing that it outperforms existing methods in terms of learning speed and learning scaling behavior, and in terms of size of the model, while it guarantees a comparable prediction accuracy.

1. Introduction

The nearest neighbor decision rule (Cover & Hart, 1967) (NN rule in the following) assigns to an unclassified sample point the classification of the nearest of a set of previously classified points. For this decision rule, no explicit knowledge of the underlying distributions of the data is needed. A strong point of the nearest neighbor rule is that, for all distributions, its probability of error is bounded above by twice the Bayes probability of error (Cover & Hart, 1967; Stone, 1977; Devroye, 1981). That is, it may be said that half the classification information in an infinite size sample set is contained in the nearest neighbor. Naive implementation of the NN rule requires to store all the previously classified data, and to compare then each sample point to be classified to each stored point. In order to reduced both space and time requirements, several techniques to reduce the size of the stored data for the NN rule have been proposed (see (Wilson & Martinez, 2000; Toussaint, 2002) for a survey) referred to as training set reduction, training set condensation, reference set thinning, and prototype selection algorithms. In particular, among these techniques, *training set consistent* ones, aim at selecting a subset of the training set that classifies the remaining data correctly through the NN rule. Using a training set consistent subset, instead of the entire training set, to implement the NN rule, has the additional advantage that it may guarantee better classification accuracy. Indeed, (Karaçali & Krim, 2002) showed that the VC dimension of a NN classifier is given by the number of reference points in the training set. Thus, in order to achieve a classification rule with controlled generalization, it is better to replace the training set with a small consistent subset. Unfortunately, computing a minimum cardinality training set consistent subset for the NN rule turns out to be intractable (Wilfong, 1992). Several training set consistent condensation algorithms have been introduced in literature. We point out that, among the criteria characterizing condensation methods, the learning speed one is usually neglected. But, in order to manage huge amounts of data, methods exhibiting good scaling behavior are definitively needed. In this work we present a novel *order independent* algorithm for finding a training set consistent subset for the NN rule, called FCNN rule, and compare it with existing methods. The rest of the paper is organized as follows. In Section 2, existing approaches are briefly described and compared to the approach here proposed. In Section 3, the FCNN rule is described and its main properties are stated. In Section 4, experimental results are presented together with a thorough comparison with existing methods. Finally, in Section 5, conclusions are drawn.

2. Related Works and Contribution

Starting from the seminal work of (Hart, 1968), several training set condensation algorithms have been introduced, also known as instance-based, lazy, memory-based, and case-based learners. These methods can be grouped into three categories depending on the objectives that they want to achieve (Brighton & Mellish,

Appearing in *Proceedings of the 22nd International Conference on Machine Learning*, Bonn, Germany, 2005.

2002), i.e. *competence preservation*, *competence enhancement*, and *hybrid approaches*. The goal of competence preservation methods is to compute a training set consistent subset removing superfluous instances that will not affect the classification accuracy of the training set. Competence enhancement methods aim at removing noisy instances in order to increase classifier accuracy. Finally, hybrid methods search for a small subset of the training set that, simultaneously, achieves both noisy and superfluous instances elimination. Competence enhancement and preservation methods are combined in order to achieve the same objectives of hybrid methods. While goals of enhancement and preservation methods are clearly separated, i.e. smoothing the decision boundary in the former case and computing a possibly small training set consistent subset in the latter, often it is not clear how subsets computed by hybrid methods can be related to the sets computed by the two other methods. Thus, searching for a small training set consistent subset is a *per se* relevant task, improving both classification speed and classifier accuracy (Karaçali & Krim, 2002), and computationally hard (Wilfong, 1992). Next, we first briefly describe incremental training set consistent subset methods for the NN rule and some other related work, and then point out the contributions of this work. A detailed survey of condensation methods can be found in (Wilson & Martinez, 2000; Toussaint, 2002).

The concept of a training set consistent subset was introduced by P.E. Hart (Hart, 1968) together with an algorithm, called CNN rule (for Condensed NN rule), to determine a consistent subset of the original sample set. The algorithm uses two bins, called S and T. Initially, the first sample of the training set is placed in S, while the remaining samples of the training set are placed in T. Then, one pass through T is performed. During the scan, whenever a point of T is misclassified using the content of S it is transferred from T to S. The algorithm terminates when no point is transferred during a complete pass of T. The motivation for this heuristic is that misclassified data lies close to the decision boundary. Nevertheless, the CNN rule may select points far from the decision boundary. Furthermore, the CNN is *order dependent*, that is it has the undesirable property that the consistent subset depends on the order in which the data is processed. The MCNN rule (for Modified CNN rule) (Devi & Murty, 2002) computes a training set consistent subset in an incremental manner. The consistent subset S is initially set to the empty set. During each main iteration of the algorithm, first the points S_t of the training set T misclassified by using S are selected. Then, the centroids C of the points in S_t are computed and used to classify S_t. S_t is thus partitioned into two sets, S_r and S_m of points that are correctly classified and misclassified respectively by using the centroids C. If the set S_m is empty, then the set S is augmented with the set C, otherwise S_t is set to S_r and the process is repeated until S_m becomes empty. The algorithm terminates when there are no misclassified points of T by using S. Differently from the CNN rule, the MCNN rule is order independent, that is, it always returns the same consistent subset independently on the order in which the data is processed. As claimed by the authors of the algorithm, the MCNN rule may work well for data with a gaussian distribution or that can be split up into regions with a gaussian distribution, but, in general, it is unlikely to select points close to the decision boundary. Also, during each iteration of the MCNN rule, at most m points can be added to the subset S, where m is the number of classes in T, thus the method could require a lot of iterations to converge. In order to compute a small consistent subset S of the training set T, (Karaçali & Krim, 2002) proposed the following algorithm (NNSRM for Structural Risk Minimization using the NN rule). Assume that the training set T contains only two class labels. First, all pairwise distances among points having different class labels are computed. Let $\{x_i, y_i\}$ denote the pair having the ith smallest distance, $i \geq 1$. The set S is initialized to $\{x_1, y_1\}$, i.e. to the closest points x_1 and y_1 from the two classes, and a counter k is initialized to 2. Next, until the set S misclassifies at least a point in T, the following steps are performed: if $\{x_k, y_k\}$ is not contained in S, then S is augmented with both x_k and y_k; in any case, k is set to $k+1$. Nevertheless, the method is costly. Indeed, the complexity of the NNSRM algorithm is $\mathcal{O}(|T|^3)$. Furthermore, we note that the size of the condensed set computed is sensitive to the pairs having greatest distances. Indeed, assume that two points from opposite classes in the training set are far from the other training set points, and such that their distance is greater than the distance of each other pair of the remaining points from opposite classes. Then, a training set consistent subset must contain these two points and, hence, all the training set points. Finally, we recall the RNN rule (Gates, 1972), a costly post-processing step that can be applied to every competence preservation method, the SNN rule (Ritter et al., 1975), having worst case exponential running time, the MCS algorithm (Dasarathy, 1994), involving computation of the so called nearest unlike neighbors, that intends to compute a subset close to the minimal one, and approximate optimization techniques (Liu & Nakagawa, 2001), that can be applied only to small sized data set.

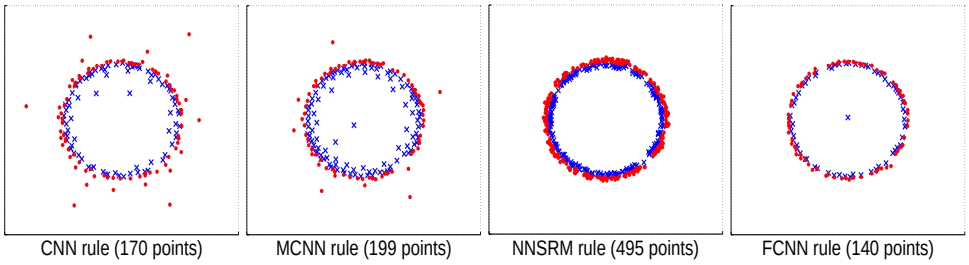

Figure 1. Example of training set consistent subsets computed by the CNN, MCNN, NNSRM, and FCNN rules.

2.1. Contribution

In this work we present a novel algorithm for the computation of a training set consistent subset for the NN rule. The algorithm, that we call Fast CNN rule (FCNN for short), works as follows. First, the consistent subset S is initialized to the centroids[1] of the classes contained in the training set T. Then, during each iteration of the algorithm, for each point p in S, a point q of T belonging to the Voronoi cell of p,[2] but having a different class label, is selected and added to S. The algorithm stops when no further point can be added to S, i.e. when T is correctly classified using S. Despite being quite simple, the FCNN rule has some desirable properties. Indeed, it is order independent, has sub-quadratic time complexity, requires few iterations to converge, and it is likely to select points very close to the decision boundary. For example, Figure 1 compares the consistent subsets computed by the CNN, MCNN, NNSRM, and FCNN rules on a training set composed by 9,000 points uniformly distributed into the unit square and partitioned in two classes by a circle of diameter 0.5. As already noted elsewhere (Wilson & Martinez, 2000), training set reduction algorithms can be characterized by their storage reduction, classification speed increase, generalization accuracy, noise tolerance, and learning speed. Among these criteria, the learning speed one is usually neglected. But, in order to be practicable on large training sets or in knowledge discovery applications requiring a learning step in their cycle, the method should exhibit good learning behavior. The contribution of this work can be summarized as follows. (*i*) We present a novel *order independent* algorithm, called FCNN rule, for the computation of a training set consistent subset for the NN rule. It is the second proposal of an order independent algorithm after the MCNN rule (Devi & Murty, 2002). (*ii*) We prove that the FCNN rule has worst case sub-quadratic time requirements, and describe an implementation exploiting the triangular inequality that sensibly reduces the worst case computational cost. (*iii*) We compare the FCNN rule with state of the art competence preservation algorithms on large and high dimensional training sets, showing that the FCNN rule outperforms existing methods in terms of learning speed and learning scaling behavior, and in terms of size of the model, while it guarantees a comparable prediction accuracy.

[1]Given a set S of points having the same class label, the centroid of S is the point of S which is closest to the geometrical center of S.

[2]The Voronoi cell of point $p \in S$ is the set of all points that are closer to p than to any other point in S.

3. Algorithm

We first give some preliminary definitions. In the following we denote by T a labelled training set from a metric space with distance metrics d. Let p be an element of T. We denote by $nn(p,T)$ the nearest neighbor of p in T according to the distance d. We denote by $l(p)$ the label associated to p. Given a point q, the NN rule $\mathrm{NN}(q,T)$ assigns to q the label of the nearest neighbor of q in T, i.e. $\mathrm{NN}(q,T) = l(nn(q,T))$. A subset S of T is said to be a *training set consistent subset of* T if, for each $p \in T$, $l(p) = \mathrm{NN}(p,S)$. Let S be a subset of T, and let p be an element of S. We denote by $\mathrm{Vor}(p,S,T)$ the set $\{q \in T \mid \forall p' \in S, \mathrm{d}(p,q) \leq \mathrm{d}(p',q)\}$, that is the set of the elements of T that are closer to p than to any other element p' of S. Furthermore, we denote by $\mathrm{Voren}(p,S,T)$ the set $\{q \in \mathrm{Vor}(p,S,T) \mid l(q) \neq l(p)\}$. We denote by $Centroids(T)$ the set containing the centroids of each class label in T. The following Theorem states the property exploited by the FCNN rule in order to compute a training set consistent subset.

Theorem 3.1 S is a training set consistent subset of T for the NN rule iff for each element p of S, $\mathrm{Voren}(p,S,T)$ is empty.

3.1. The FCNN Rule

The algorithm FCNN rule is shown in Figure 2. It initializes the consistent subset S with a seed element from each class label of the training set T. In particu-

Algorithm FCNN rule
Input: A training set T;
Output: A training set consistent subset S of T;
Method:
  $S = \emptyset$;
  $\Delta S = Centroids(T)$;
  while $(\Delta S \neq \emptyset)$ {
    $S = S \cup \Delta S$;
    $\Delta S = \emptyset$;
    for each $(p \in S)$
      $\Delta S = \Delta S \cup \{rep(p, \text{Voren}(p, S, T))\}$;
  }
  return(S);

Figure 2. The FCNN rule.

lar, the seeds employed are the centroids of the classes in T. The algorithm is incremental: during each iteration the set S is augmented until the stop condition, given by Theorem 3.1, is reached. For each element of S, a representative element of $\text{Voren}(p, S, T)$ w.r.t. p, denoted as $rep(p, \text{Voren}(p, S, T))$ in Figure 2, is selected and inserted into S. Given $p \in T$ and a subset $X \subseteq T$, the representative $rep(p, X)$ of X w.r.t. p can be defined in different ways. We investigate the behavior of two different definitions for $rep(p, X)$. The first definition, we call FCNN1 the variant of the FCNN rule using this definition, selects the nearest neighbor of p in X, that is $rep(p, X) = nn(p, X)$. The second definition, we call FCNN2 the variant using it, selects the class centroid in X closest to p, that is $rep(p, X) = nn(p, Centroids(X))$. If, during an iteration, no new element can be added to S, then, by Theorem 3.1, S is a training set consistent subset of T, and the algorithm terminates returning the set S. Figure 4 reports an example of execution of the FCNN1 rule. The data set considered is composed by 2,000 points on the plane. Half of the points belong to one of two concentric spirals representing two distinct classes. In this case, the algorithm performs nine iterations and returns a subset composed by 54 points (the FCNN2 rule on the same data set performs ten iterations and computes a subset composed by 74 points). It is clear from these figures that the FCNN rule rapidly converges to a solution. Indeed, the number of points contained in the subset could double at the end of each iteration. This is due to the fact that, for each point p such that $\text{Voren}(p, S, T)$ is not empty, a new point is added to the set S. The following theorem states the main properties of the algorithm.

Theorem 3.2 *The algorithm FCNN rule* (i) *terminates in a finite time,* (ii) *computes a training set consistent subset, and* (iii) *is order independent.*

Figure 3 shows the implementation of the FCNN1 rule.

Algorithm FCNN1 rule
Input: A training set T;
Output: A training set consistent subset S of T;
Method:
  for each $(p \in T)$
    $nearest[p]$ = undefined;
  $S = \emptyset$;
  $\Delta S = Centroids(T)$;
  while $(\Delta S \neq \emptyset)$ {
    $S = S \cup \Delta S$;
    for each $(p \in S)$
      $rep[p]$ = undefined;
    for each $(q \in (T - S))$ {
      for each $(p \in \Delta S)$
        if $(\text{d}(nearest[q], q) < \text{d}(p, q))$
          $nearest[q] = p$;
      if $((l(q) \neq l(nearest[q]))$ && $(\text{d}(nearest[q], q)$
        $< \text{d}(nearest[q], rep[nearest[q]]))$
          $rep[nearest[q]] = q$;
    }
    $\Delta S = \emptyset$;
    for each $(p \in S)$
      if ($rep[p]$ is defined) $\Delta S = \Delta S \cup \{rep[p]\}$;
  }
  return(S);

Figure 3. The FCNN1 rule.

The algorithm maintains in the array *nearest*, for each training set point q, the closest point $nearest[q]$ of q in the set S, and in the array *rep*, for each point p in S, its current representative $rep[p]$ of the misclassified points lying in the Voronoi cell of p. During each iteration, the array *nearest* and *rep* must be updated. Let ΔS be the set of points added to the set S at the beginning of the current iteration. To update the array *nearest*, it is sufficient to compare the training set points in $(T - S)$ with the points in the set ΔS, as the nearest neighbors in $S - \Delta S$ of the points in $(T - (S - \Delta S))$ where computed in the previous iterations, and are already stored in *nearest*. After having computed the closest point $nearest[q]$ in ΔS, and hence in S, of a point q in $(T - S)$, the array *rep* can be updated efficiently as follows. If the label of q is different from the label of $nearest[q]$, then q is misclassified. In this case, if the distance from $nearest[q]$ to q is less than the distance from $nearest[q]$ to its current associated representative $rep[nearest[q]]$, then $rep[nearest[q]]$ is set to q. The following theorem states an upper bound to the complexity of the FCNN rule.

Theorem 3.3 *The FCNN rule requires at most* $|T| \cdot |S|$ *distance computations using* $\mathcal{O}(|T|)$ *space.*

The FCNN2 has a very similar implementation. The only difference lies in the update of the array *rep*. Indeed, in order to determine the centroids of the misclassified points of each Voroni cell induced by the

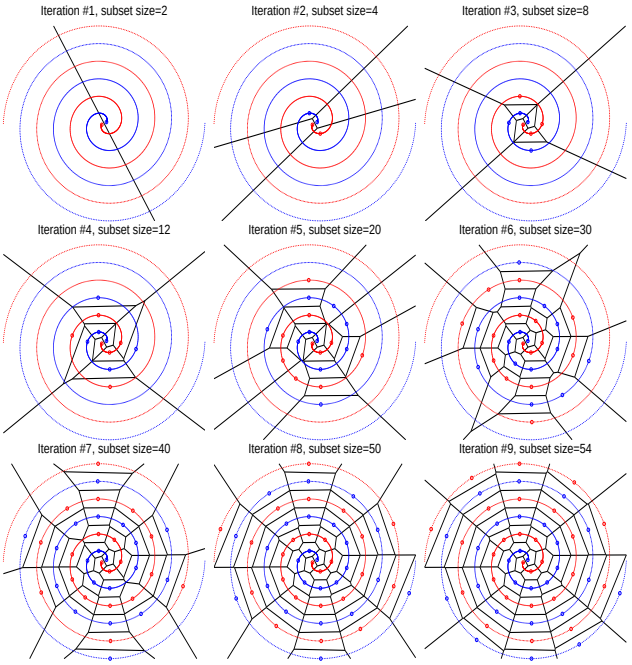

Figure 4. Example of execution of the FCNN1 rule.

points in S, the FCNN2 rule performs at the end of each main iteration a supplemental training set scan.

3.2. Exploiting the Triangular Inequality

In a metric space the FCNN rule can take advantage of the triangular inequality in order to avoid the comparison of each point in $(T - S)$ with each point in ΔS. During the generic ith main iteration, for each $p \in S_{i-1}$, the Voronoi cell $\mathrm{Vor}(p, S_{i-1}, T)$[3] is visited, and the points $q \in \mathrm{Vor}(p, S_{i-1}, T)$ are compared with the points in ΔS_i. Before starting the comparison, the distances among the point p and the points in ΔS_i are computed, and the points in ΔS_i are sorted in order of increasing distance from p. Then, instead of comparing each point q in $\mathrm{Vor}(p, S_{i-1}, T)$ with each point in ΔS_i, each q is compared with the points in ΔS_i having distance from p less than twice the distance from q and p. Indeed, by the triangular inequality, they are all and the only points of ΔS_i candidate to be closer to q than p. In the worst case, this implementation of the FCNN rule requires $|T| \cdot |S| + \sum_{i<j} |\Delta S_i| \cdot |\Delta S_j| \cdot \log |\Delta S_j|$ distance computations, i.e. slightly worse than the upper bound of Theorem 3.3, and has no additional space requirements. Nevertheless, the last implementation guarantees great savings in terms of distances computed, and definitively outperforms the previous one.

[3]Partitioning of points in Voronoi cells is induced by the array *nearest*.

Data set name	*Size*	*Features*	*Classes*
Census	190,495	13	2
Chessboard	64,000	2	2
Forest Cover Type	100,000	54	7
DARPA	458,301	22	5
Shuttle	43,500	9	7
Spiral	1,000,000	2	2

Table 1. Data sets used in the experiments.

4. Experimental Results

In this section we present a number of experiments performed using the FCNN rule, and a thorough comparison with the CNN and MCNN methods. The NNSRM rule is impracticable on large training sets, hence we do not consider it further. As for the FCNN rule, we used the algorithm exploiting the triangular inequality described in Section 3.2. As for the MCNN rule, we augmented the method with the technique exploiting the triangular inequality described in Section 3.2. As for the CNN rule, we avoided repeated distance computations. The data set employed are summarized in Table 1, and briefly described next. *Census* is composed by data extracted from the census bureau database[4]. *Chessboard* is a synthetic data set composed by points uniformly distributed into the unit square and grouped in two classes representing cells of a 8×8 chessboard. *Forest Cover Type* contains data from the US Forest Service[5]. The *DARPA* 1998 intrusion detection data consists of network connection records of several intrusions[6]. *Shuttle* was used in the European StatLog project[5]. *Spiral* is a synthetic data set composed by two concentric spirals representing two distinct classes.

Execution time, size and accuracy. Table 2 compares the FCNN1, FCNN2, MCNN, and CNN rules on the data sets above described, using the Euclidean distance. The table reports execution time[7], number of iterations performed, empirical complexity, size of the condensed set, and classification accuracy obtained using the condensed set. In order to estimate the classification accuracy of each rule, we used 10-fold cross-validation. The *empirical complexity* is the ratio $\log D / \log |T|$, where D denotes the number of distances computed by the method, and provides an estimate of its computational complexity. Although it provides a short summary, it is not sufficient to characterize the effective effort of the method, since the execution time is influenced also by the number of it-

[4]`www.census.gov/ftp/pub/DES/www/welcome.html`.
[5]`www.kdd.ics.uci.edu`.
[6]`www.ll.mit.edu/IST/ideval/index.html`.
[7]We ran the experiments on a Pentium IV 2.66GHz based machine.

Data set name	Method	Execution time [sec]	Iterations	Empirical complexity	Size	Accuracy [%]
Census	FCNN1	240.33	36	1.69	**24,614**	93.06
	FCNN2	**185.42**	26	**1.62**	25,248	93.05
	MCNN	4,283.73	19,251	1.86	27,666	93.02
	CNN	1,818, 77	**6**	1.84	27,836	**93.11**
	training set	–	–	–	171,445	94.61
Chessboard	FCNN1	4.17	54	1.56	**3,082**	**97.14**
	FCNN2	**3.03**	20	**1.47**	3,707	**97.14**
	MCNN	84.36	2,019	1.76	3,995	97.03
	CNN	28.30	**5**	1.75	3,812	97.11
	training set	–	–	–	57,599	98.75
Forest Cover Type	FCNN1	184.34	33	1.67	**15,972**	90.68
	FCNN2	**178.06**	24	**1.65**	16,255	**90.90**
	MCNN	2,341.81	7,938	1.86	16,560	90.66
	CNN	1,294, 25	**5**	1.84	16,920	90.69
	training set	–	–	–	90,000	92.82
DARPA	FCNN1	63.09	245	1.44	**3,252**	**99.22**
	FCNN2	**27.01**	23	**1.37**	3,803	99.20
	MCNN	583.91	2,566	1.56	4,275	99.21
	CNN	732.81	**14**	1.64	4,078	99.20
	training set	–	–	–	412,470	99.31
Shuttle	FCNN1	**0.35**	21	1.36	**225**	**99.75**
	FCNN2	0.67	16	**1.35**	257	99.69
	MCNN	1.67	144	1.45	286	99.71
	CNN	1.66	**4**	1.53	280	99.71
	training set	–	–	–	39,150	99.77
Spiral	FCNN1	**2.39**	9	**1.21**	**57**	**100.00**
	FCNN2	3.75	10	1.22	80	**100.00**
	FMCNN	22.01	46	1.37	88	**100.00**
	CNN	6.30	**2**	1.32	77	**100.00**
	training set	–	–	–	900,000	100.00

Table 2. Experimental results.

erations and by the number of training set passes per iteration. Next, we comment on the results shown in Table 2. As for the number of iterations, we observe that the CNN rule requires few iterations to converge, the MCNN rule requires a lot of iterations, while the FCNN1 and FCNN2 require a little number of iterations. The FCNN rule requires more iterations than the CNN rule, but notably less than the MCNN rule. We recall that the CNN and the FCNN1 rule perform one training set pass per iteration, that the FCNN2 rule performs two training set passes per iteration, and that the MCNN rule performs several training set passes per iteration. As for the empirical complexity, the FCNN1 and FCNN2 rules outperform both the MCNN and the CNN rule. The FCNN2 rule performs in some cases slightly better than the FCNN1 rule, while the CNN and the MCNN rule seem comparable. As for the execution time, the FCNN1 and FCNN2 rules outperform both the two other rules by at least an order of magnitude. This is a consequence of the fact that the FCNN rule has a lower empirical complexity, requires few iterations to converge, and one (or two) training set pass per iteration. As for the size of the training set consistent subset, the set computed by the FCNN1 rule was always the smallest. Finally, as for the classification accuracy, the FCNN rules achieves almost always the best values, though the quality of classification is about the same for all the methods.

Learning behavior. Figure 5 shows the learning behavior of the FCNN1, FCNN2, MCNN, and CNN rules on the data sets *Census*, *Forest Cover Type*, and *DARPA*. The curves show (from left to right) the accuracy of the subset S_i computed during the ith iteration of the algorithms in classifying the overall training set T, the number of points $|\Delta S_i|$ added to S_i during each iteration, the size $|S_i|$ of the subset S_i versus the normalized iteration number $\frac{i}{i_{\max}}$, and the distances computed during each iteration. As for the accuracy (first column), we note that the FCNN2 rule smoothly converges to consistency, while the curve of the FCNN1 rule swings during the initial iterations (notice that the graphic of the DARPA data set is in logarithmic scale). We note also that the area under the curve of the FCNN2 rule is always the greatest. As for the size of ΔS_i (second column), interestingly the FCNN rule

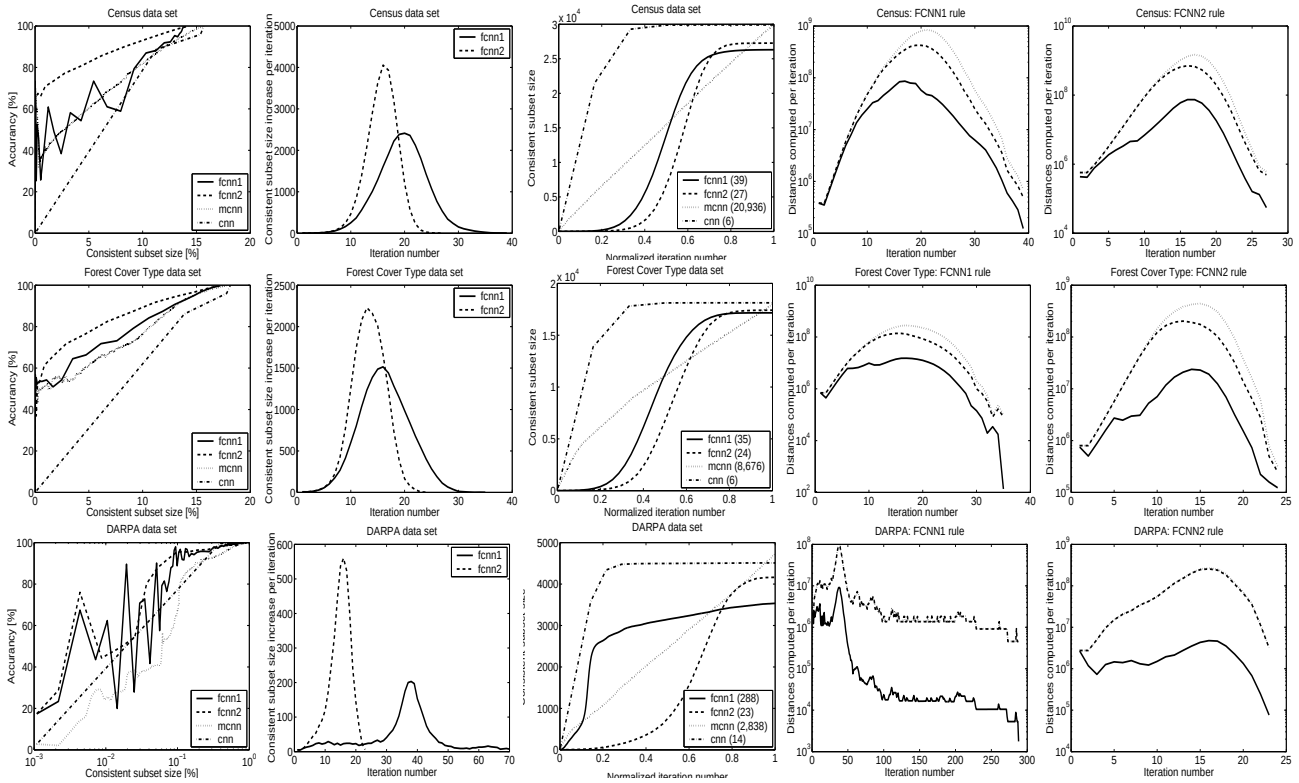

Figure 5. Learning behavior.

follows a gaussian-like distribution. Though the area under the curves of the FCNN1 and FCNN2 rules is nearly identical, we note that the FCNN1 rule reaches its maximum later, and hence requires more iterations to converge. Analogous considerations hold for the course of $|S_i|$ (third column). The number of iterations performed by the methods is reported in the figure legend. We note that the FCNN1 rule always computes the smallest consistent subset. The curve of the MCNN rule is a straight line as the number of points added during each iteration is almost constant and given by the number of class labels in the training set, the curve of the CNN rule is a step as almost all the points are selected during the first and second iteration, while the curves of the FCNN rule resemble a sigmoid. Finally, as for the distances computed (fourth and fifth columns), solid lines represent the number of distances actually calculated by the FCNN rule exploiting the triangular inequality, while dotted lines represent the worst case cost of the same algorithm. Dashed lines represent the distances required when the triangular inequality is not exploited. It is clear from these figures that great savings are obtained, since the area between the solid and the dashed curve is at least one order of magnitude greater than the area under the solid curve.

Scaling analysis. Figure 6 shows the scaling analysis on the training sets *Census*, *Forest Cover Type*, and *DARPA*. As for the execution time (first column), both the FCNN1 and FCNN2 rules clearly outperform the other two rules. The CNN rule performs better than the MCNN rule on the first two data sets. We note that the number of iterations performed (second column) by the FCNN and the CNN rules is almost constant with the training set size, while the iterations performed by the MCNN rule increases of some order of magnitude with the training set size. As for the empirical complexity (third column) the FCNN rule performs better. The FCNN2 rule was always the best. On the *Census* data set the empirical complexity increases with the size, while on the *DARPA* data set it decreases, for all methods. *Forest Cover Type* is hard to manage by the CNN and the MCNN rules, whose empirical complexity is constant, while it less difficult for the FCNN rule, whose empirical complexity decreases with the data set size. Finally, as for the consistent subset size (fourth column), the curves of the various methods have the same behavior, but it is worth to note that on these data sets the FCNN rules computes the smallest training set consistent subsets.

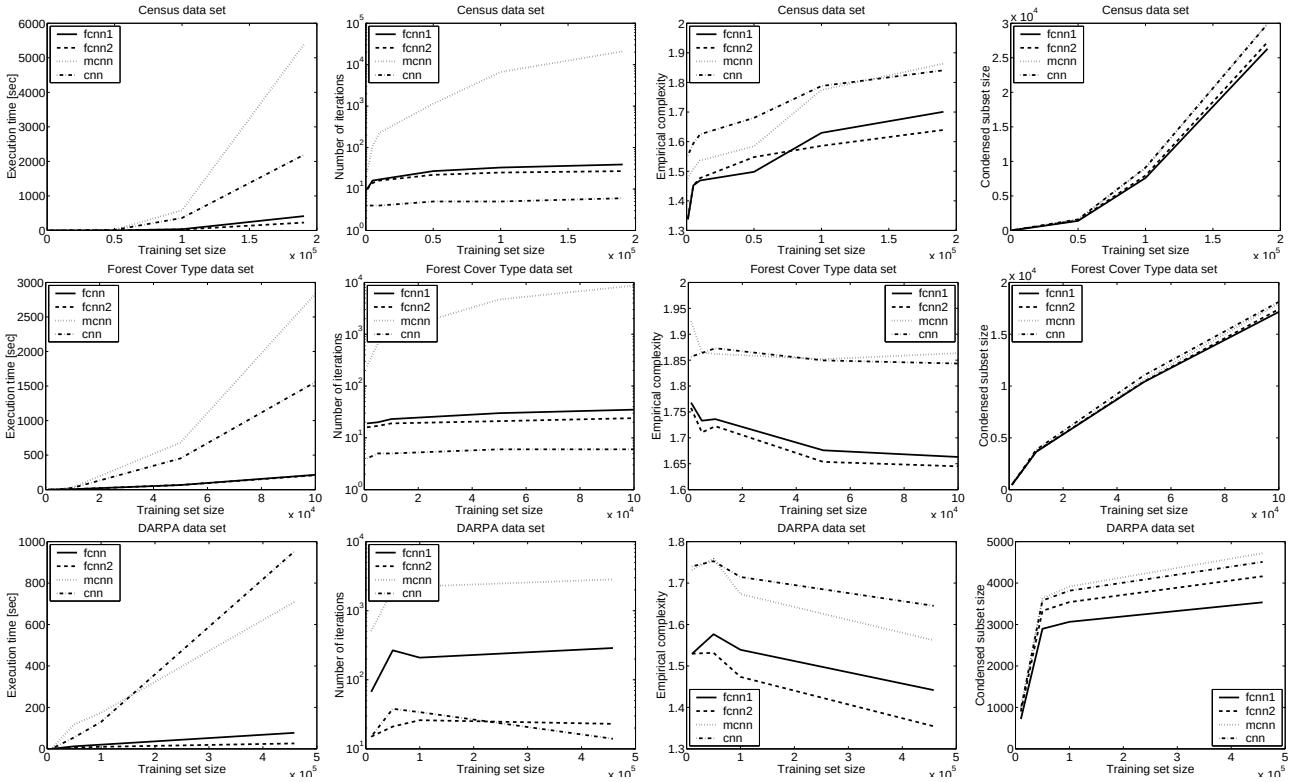

Figure 6. Scaling analysis.

5. Conclusions

We presented a novel order independent method for computing a training set consistent subset for the NN rule and compared it with existing state of the art competence preservation methods. The observed superior learning speed of the new method is substantiated by the learning behavior comparison. This work can be extended in several ways, e.g., studying the impact of different metrics on the FCNN rule, and the behavior of FCNN-based hybrid methods.

References

Brighton, H., & Mellish, C. (2002). Advances in instance selection for instance-based learning algorithms. *Data Mining and Know. Discovery*, *6*, 153–172.

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Trans. on Inform. Th.*, *13*, 21–27.

Dasarathy, B. (1994). Minimal consistent subset (mcs) identification for optimal nearest neighbor decision systems design. *IEEE Trans. on Sys. Man. Cybernet.*, *24*, 511–517.

Devi, F., & Murty, M. (2002). An incremental prototype set building technique. *Pat. Recognition*, *35*, 505–513.

Devroye, L. (1981). On the inequality of cover and hart. *IEEE Trans. on Pat. Anal. and Mach. Intel.*, *3*, 75–78.

Gates, W. (1972). The reduced nearest neighbor rule. *IEEE Trans. on Inform. Th.*, *18*, 431–433.

Hart, P. (1968). The condensed nearest neighbor rule. *IEEE Trans. on Inform. Th.*, *14*, 515–516.

Karaçali, B., & Krim, H. (2002). Fast minimization of structural risk by nearest neighbor rule. *IEEE Trans. on Neural Networks*, *14*, 127–134.

Liu, C., & Nakagawa, M. (2001). Evaluation of prototype learning algorithms for nearest-neighbor classifier in application to handwritten character recognition. *Pat. Recognition*, *34*, 601–615.

Ritter, G., Woodruff, H., Lowry, S., & Isenhour, T. (1975). An algorithm for a selective nearest neighbor decision rule. *IEEE Trans. on Inform. Th.*, *21*, 665–669.

Stone, C. (1977). Consistent nonparametric regression. *Annals of Statistics*, *8*, 1348–1360.

Toussaint, G. (2002). Proximity graphs for nearest neighbor decision rules: Recent progress. *Proc. of the Symp. on Computing and Stat.*. Montreal, Canada.

Wilfong, G. (1992). Nearest neighbor problems. *Int. J. of Computational Geometry & Applications*, *2*, 383–416.

Wilson, D., & Martinez, T. (2000). Reduction techniques for instance-based learning algorithms. *Machine Learning*, *38*, 257–286.